

中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生

数学建模竞赛

学校	华南师范大学
参赛队号	26105740044
队员姓名	1.林佳坤
	2.宋元明
	3.彭甚偲

中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生

数学建模竞赛

题 目： 用于算力约束的大模型最优 Loss 资源配置建模

摘 要：

关键词： 数据质量评价, 领域配比，Chinchilla, 算力约束，粒子群算法

目 录

1. 问题重述和问题分析 3
2. 模型假设和符号说明 5
3. 问题一：数据质量评分 Q、冲突修正和配比 p 6
   3.1 Pearson 相关系数和领域指标分类 6
   3.2 正向化、PCA、熵权、TOPSIS 评分——联合计算Q值 8
   3.3 冲突指标对和冲突分析 16
   3.4 对数 L-p模型和最优p 19
4. 问题二：广义标度率LN, D, Q, p的显示表达 25
   4.1 指数项Q和分配项p 25
   4.2 岭回归模型和求解 25
5. 问题三：算力约束下的最优 Loss 配置 29
   5.1 多目标约束问题 29
   5.2 粒子群算法和求解 29
6. 问题四：预测和 benchmark 31
   6.1 问题四的模型 31
   6.2 问题四的预测和结果 32
7. 模型评价 32
   7.1 粒子群算法的灵敏度分析 32
   7.2 模型缺点 34

参考文献 34

附录 A 附件清单 35

附录 B 核心源码 35

1. 问题重述和问题分析

1.1 问题背景和提出

以大语言模型为代表的人工智能技术进展迅速，但其训练与生成始终受到资源制约。类似人类的学习，大语言模型类似于培养高质量学生：参数量 N 如脑容量，数据量 D 如阅读量，计算量 C 如教育经费，数据质量 Q 如教材质量，领域配比 p 如各科课时分配。标度律刻画损失随规模变化的经验关系，常见形式为

$$L(N,D) = E + A{\cdot}N^{-\alpha} + B{\cdot}D^{-\beta}$$

其中E为不可约损失，(A,B,α,β)为待估参数，$L-Loss$指验证集交叉熵；模型能力还通过下游多维基准衡量，二者相关但不等价。随着对高质量语料需求增长，干净数据日益稀缺，数据污染严重，数据墙问题突出。2026 年 2 月《Nature》的一篇研究[1]指出，在引文准确率、降低幻觉等任务上，仅 80 亿参数的 OpenScholar-8B 可优于 GPT-4o，表明数据质量 Q 与领域配比 p 对模型能力有不可忽视的作用。然而，扩大模型规模、提升数据质量、延长上下文长度都需算力，而算力有限；上下文越长，注意力计算与 KV 缓存等序列开销越高，会挤压用于参数、数据或质量提升的预算。调整领域配比虽不增加总算力，却会改变算力在各领域间的分配效果。因此，N、D、Q、上下文长度与 p 之间存在复杂取舍，需在有限算力、能耗和成本下综合优化，才能提升训练效率、下游表现与生成推理性价比。赛题提出如下问题

问题一：

（1）建立样本级、语料级或领域级的数据质量评分Q模型，指标需“越高越好”。使用数据集为训练集 A1 验证集 A2A3，给出领域级Q；

（2）在差异化指标对同一样本给出显著不一致评价时定义冲突，并消解冲突，在验证集验证结论；

（3）找到配比权重$\boldsymbol{p}$和 Loss 的定量模型，使用数据集为 A4-A15，外推集为 A6-A11 讨论外推稳健性。

问题二：

找到广义标度率$L(N,D,Q,\boldsymbol{p})$的显式表达式，使用数据集为 B1 - B12。

问题三：

加入算力约束 C，找到最优 Loss 的资源配置方案。算力消耗由以下三部分组成。

1. 基础训练开销：$C_{\text{train}} = 6ND$（Chinchilla 近似），其中 N 为参数个数，D 为训练 Token 数。

2. 数据质量提升开销。设基线质量为 Q_0（可由附件 A 的质量评分或合理假设给出），将质量由 Q_0 提升至 $Q \in (0,1]$ 需要额外算力；若采用增量成本形式，可按下式计算

$$C_Q = D\,[g(Q) - g(Q_0)]_+,$$

其中 $g(Q)$ 可从指数型、幂函数型、对数渐进型三类中选取或改进，具体形式与参数见附录 B。

3.长文本注意力开销：$C_{\text{attn}} = \eta N D L_{\text{ctx}}$，其中 $\eta = 2 \times 10^{-4}$，L_{ctx} 为上下文窗口长度（外生给定，取值依据 C7）。上下文长度越大，所需处理的序列位置越多，注意力与缓存等开销越高；本问使用上式作为简化算力代理。

问题四：

预测模型的发展与演化，主要使用多维 Benchmark 得分进行评价结果。

四问构成一条递进主线：问题一刻画数据本身的质量与配比，问题二把数据因素纳入标度律，问题三在算力预算下求解最优配置，问题四用历史评测数据检验规律并预测能力前沿；前一问的输出是后一问的输入。

1.2 问题分析

针对问题一，我们设计了领域分配评价表

2. 模型假设和符号说明

2.1 模型假设

1.假设标度律只受题给参数影响

2.假设源样本数据的给出均无误

2.2 符号说明

序号	符号	说明	单位
1	D, d	数据，样本数据	Token
2	Q	数据质量评分	
3	$\boldsymbol{p}$	领域配比	
4	***L-Loss***	交叉熵损失	
5	N	模型参量	B, $1\mathrm{B} = 10^9$
6	C	算力	FLOPS
7	N, O, P	指标取向评价	
8	A, B, α, β	参数	

3. 问题一：数据质量评分 Q、冲突修正和配比 p

问题一主要在于给出Q和$\boldsymbol{p}$的确切计算方式。我们先对指标进行相关系数分析，然后人工划分各领域的对应指标并给出统一指标取向，消除部分大差异指标对样本的不一致评价冲突，使用 PCA+熵权+TOPSIS 打分联合计算出样本级Q，并选取冲突指标对进行冲突分析。接着用线性负对数模型最小二乘出$L(p)$的表达式参数，之后外推讨论。

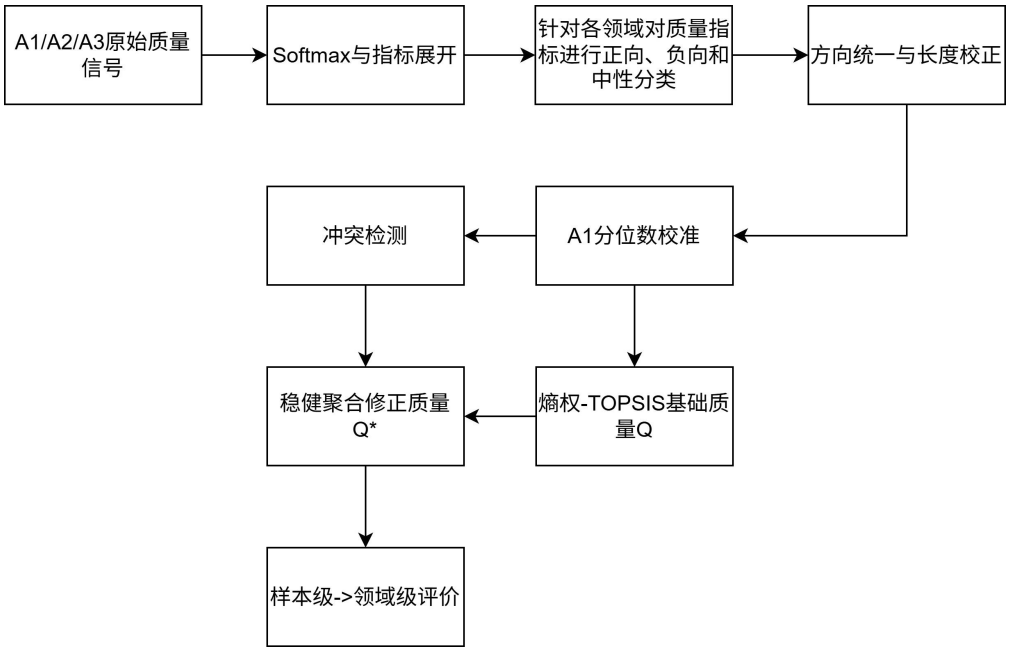


图 1 计算Q和修正Q的整体流程

3.1 Pearson 相关系数和领域指标分类

Pearson 相关系数是用协方差除以随机变量标准差的积得到

$$\rho_{XY} = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

用以衡量变量之间是否具有线性关系。其各个部分的组成为

$$\rho_{XY} = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \overline{y})^2}}$$

需要注意的是，不相关不一定就独立，相关系数作为必要条件存在。以 A1 为样本数据将四维的 Qurater 指标进行拆分计算得到的 25 维指标热力图，见图 1。观察发现，大多数指标不具备显著相关性，而 dsir_books / dsir_wiki / dsir_math 三项出现显著现象相关，DSIR 全称为 Data Selection with Importance Resampling 是一种重要的性采集方法。

由于样本的选取按照不同域（domain）进行划分，而不同域中指标存在差异性的正负取向，我们首先需要按照领域的属性进行人工评级，取向定义为：

（i）正向（P）表示该指标值高与该来源的典型特征一致或有利于该来源的数据质量；

（ii）负向（N）表示指标值高与该来源特征相悖或不利于质量；

（iii）中性（O）表示无明显偏向或影响不大，详见表 1。

尽管各领域在部分指标上存在正负向差异，但绝大多数指标（如词汇丰富度、流畅度、写作风格、整洁性、教育价值等）对所有领域均呈现正向关联，这意味着它们能够跨领域地反映文本的基础质量与规范性。然而，即使已按领域分配指标，一些固有冲突仍无法消除，例如相近指标可能给出不一致的评价：同一个文档可能在词汇多样性上得分很高，却在重复性指标上表现不佳，导致评分内部矛盾。为此，我们挑选固有冲突指标对（如词汇丰富度与覆盖率、教育价值与专业性需求等），并设计修正公式将原始评分 Q 值进行校准。

表 1 指标语义按领域人工分类表，P 为正向，N 为负向，O 为中性

序号	指标	ArXiv	Book	C4	CommonCrawl	GitHub	StackExchange	Wikipedia
1	文档句子数量	P	P	O	O	N	P	P
2	单词平均字符长度	P	O	O	O	N	O	O
3	正常标点结尾行比例	P	P	O	O	N	P	P
4	大写字母占比	O	N	N	N	N	N	N
5	数字字符占比	P	O	N	N	O	P	O
6	不同单词占比	P	P	P	P	P	P	P
7	词频信息熵	P	P	P	P	P	P	P
8	文档词数	P	P	O	O	N	P	P
9	无字母单词占比	O	N	N	N	P	O	N
10	最频繁二元组覆盖字符比	N	N	N	N	N	N	N
11	最频繁三元组覆盖字符比	N	N	N	N	N	N	N
12	与书籍语料的分布相似度	P	P	N	N	O	P	P
13	与 Wiki 语料的分布相似度	P	P	N	N	O	P	P
14	与数学语料的分布相似度	P	P	N	N	O	P	P
15	广告检测二分类 logit	N	N	P	P	N	N	N
16	流畅度二分类 logit	P	P	P	P	P	P	P
17	Fine Web-Edu 教育价值评分	P	P	N	N	O	P	P
18	Qu Rating 写作风格	P	P	P	P	P	P	P
19	Qu Rating 所需专业知识	P	O	N	N	P	P	O
20	Qu Rating 事实与知识含量	P	P	N	N	O	P	P
21	Qu Rating 教育价值	P	P	N	N	O	P	P
22	Modern BERT 专业性	P	O	N	N	P	P	O
23	Modern BERT 可读性	P	P	P	P	O	P	P
24	Modern BERT 推理复杂度	P	O	N	N	P	P	O
25	Modern BERT 整洁性	P	P	P	P	P	P	P

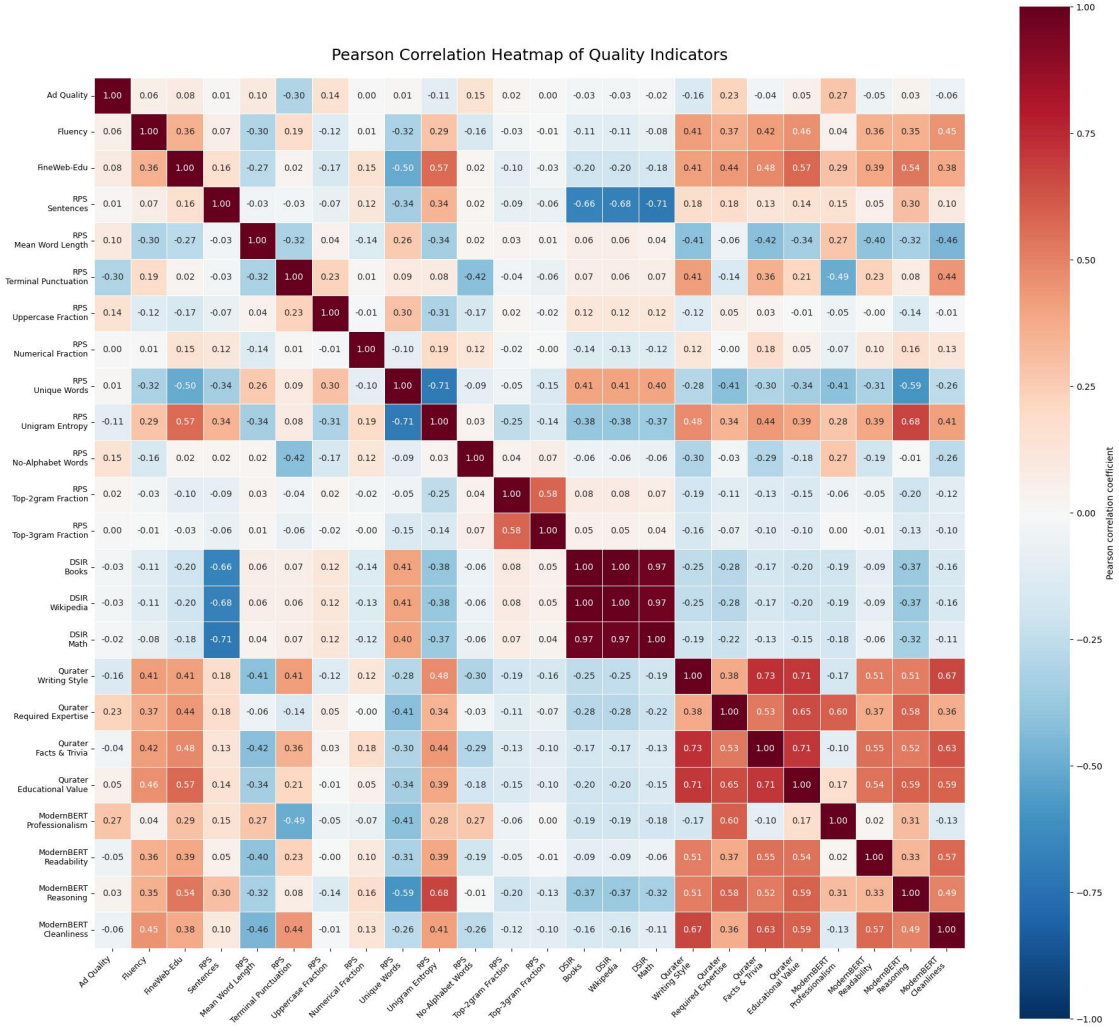


图 2 数据指标的 Perason 相关系数，拆分 Qurater 共 25 维

3.2 正向化、PCA、熵权、TOPSIS 评分——联合计算Q值

设单一样本d在不同指标下分量为d_i，即

$$d = (d_1, d_2, \ldots, d_m)$$

则数据集可以写成矩阵

$$D = \begin{pmatrix} d_{11} & d_{12} & \cdots & d_{1m} \\ d_{21} & d_{22} & \cdots & d_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ d_{n1} & d_{n2} & \cdots & d_{nm} \end{pmatrix}$$

正向化

由于语义和领域的存在，样本数据在不同指标的量纲与变化方向上互不一致，因此须先作正向化处理，把全部指标统一为越大越优，再作标准化处理：

（i）当第 j 项指标为正向指标（P），即指标值越大越优时，正向化后仍取原值，正向化公式为

$$x_{ij} = d_{ij}$$

（ii）当第 j 项指标为负向指标（N），即指标值越小越优时，采用线性反向操作，使最小值与最大值互换位置，取向公式变为

$$x_{ij} = max_i d_{ij} + min_i d_{ij} - d_{ij}$$

（iii）当第 j项指标为中性指标（O），即指标取值以某一理想值 d_j^0 为最佳时，采用偏离度反向变换，其正向化公式为

$$x_{ij} = 1 - \frac{|d_{ij} - d_j^0|}{max_i|d_{ij} - d_j^0|}$$

经正向化处理后，数据集矩阵D 化为同向矩阵X，即

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}$$

接下来消除量纲不同带来的数据差异过大影响，对同向矩阵作 z分数标准化，令

$$y_{ij} = \frac{x_{ij} - \overline{x}_j}{s_j}$$

式中第 j项指标的样本均值与样本标准差分别为

$$\overline{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}$$

$$s_j = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \overline{x}_j)^2}$$

标准化后每一列的均值为零，平方和为 $n-1$，即

$$\sum_{i=1}^{n} y_{ij} = 0$$

$$\sum_{i=1}^{n} y_{ij}^2 = n - 1$$

量纲的影响由此被消除，记标准化后的矩阵为

$$Y = \begin{pmatrix} y_{11} & y_{12} & \cdots & y_{1m} \\ y_{21} & y_{22} & \cdots & y_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ y_{n1} & y_{n2} & \cdots & y_{nm} \end{pmatrix}.$$

PCA

主成分分析的作用在于消除指标之间的多重共线性，并把 m 个相关指标压缩为少数互不相关的综合指标。为此先计算标准化数据的相关系数矩阵

$$R=\begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1m} \\ r_{21} & r_{22} & \cdots & r_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ r_{m1} & r_{m2} & \cdots & r_{mm} \end{pmatrix}$$

相关系数矩阵的元素表达式为

$$r_{jk}=\frac{1}{n-1}\sum_{i=1}^{n} y_{ij}\, y_{ik}$$

由于各列标准化后的均值为零，上式与 Pearson 相关系数的通常定义一致，且 R 是实对称半正定矩阵，主对角线元素全为 1，迹 $\mathrm{tr}(R)=m$。对 R 作正交分解对角化

$$R=V\Lambda V^{T}$$

中间的特征值矩阵为

$$\Lambda=\begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_m \end{pmatrix}$$

各个特征值满足从大到小排列

$$\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_m \geq 0$$

自然有特征值关系

$$\mathrm{tr}(R)=\sum_{j=1}^{m}\lambda_j = m$$

而向量矩阵

$$V=\begin{pmatrix} v_{11} & v_{12} & \cdots & v_{1m} \\ v_{21} & v_{22} & \cdots & v_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ v_{m1} & v_{m2} & \cdots & v_{mm} \end{pmatrix}$$

是特征向量矩阵，第j列即为第j个主成分所对应的特征向量

$$v_j=\left(v_{1j}, v_{2j}, \cdots, v_{mj}\right)^{T}$$

这样第j个主成分的方差贡献率为

$$\alpha_j=\frac{\lambda_j}{\sum_{k=1}^{m}\lambda_k}=\frac{\lambda_j}{m}$$

反应了该主成分所解释的方差份额。前 q 个主成分的累计方差贡献率为

$$A_p = \sum_{j=1}^{p} \alpha_j$$

一般取使下式成立的最小整数p，以保证降维之后仍保留原始数据 85% 以上的信息。即

$$p := A_p \geq 0.85$$

将标准化数据向保留的特征方向投影，即得主成分得分

$$f_{ij} = \sum_{k=1}^{m} v_{kj}\, y_{ik}$$

式中 v_kj 为特征向量v_j的第 k 个分量。记得分矩阵为

$$F = \begin{pmatrix} f_{11} & f_{12} & \cdots & f_{1q} \\ f_{21} & f_{22} & \cdots & f_{2q} \\ \vdots & \vdots & \ddots & \vdots \\ f_{n1} & f_{n2} & \cdots & f_{nq} \end{pmatrix}.$$

f_{ij}是第 i个样本在第 j个主成分上的取值，原来的 m 维相关评价问题由此化为 q 维互不相关的综合评价问题。

熵权

接下来用熵权法从数据自身的离散程度出发客观确定各主成分的权重：某项主成分在样本之间取值差异越大，其区分能力越强、包含的有效信息越多，相应的熵越小而权重越大。由于主成分得分可正可负，先作平移极差变换

$$\tilde{f}_{ij} = \frac{f_{ij} - min_i f_{ij}}{max_i f_{ij} - min_i f_{ij}} + \varepsilon$$

使取值非负，式中 ε 为充分小的正数，用以避免对数运算无定义。再计算第j个主成分下第i个样本的比重

$$q_{ij} = \frac{\tilde{f}_{ij}}{\sum_{i=1}^{n} \tilde{f}_{ij}}$$

由变换的非负性知 q_{ij} 非负，且各样本比重之和为 1，即

$$\sum_{i=1}^{n} q_{ij} = 1$$

据此定义第j个主成分的信息熵

$$e_j = -\, k \sum_{i=1}^{n} q_{ij} \ln q_{ij}$$

式中归一化常数取

$$k = \frac{1}{\ln n}$$

并约定 q_{ij} 为零时$q_{ij} ln q_{ij}$ 亦取零。由 $0 \leq q_{ij} \leq 1$ 及熵的极值性质可得信息熵的取值范围为

$$0 \leq e_j \leq 1$$

当各样本在该主成分上的比重完全相同时信息熵取最大值 1，即

$$q_{ij} = \frac{1}{n}$$

当信息集中于个别样本时信息熵趋近于 0

$$e_j \to 0$$

令差异系数，又称信息效用值，为

$$g_j = 1 - e_j$$

则第 j 个主成分的熵权为

$$\beta_j = \frac{g_j}{\sum_{k=1}^{p} g_k}$$

显然各熵权非负，且熵权之和为 1

$$\sum_{j=1}^{p} \beta_j = 1$$

特别地，若某主成分在所有样本上取值完全相同，则由比重恒为 $\frac{1}{n}$ 直接得到

$$e_j = 1, g_j = 0, \beta_j = 0$$

即无区分度的信息被自动剔除，这正是熵权法客观性的体现。主成分分析给出的方差贡献率 α_j反映各主成分对原始数据信息量的解释能力，熵权 β_j 反映各主成分对样本间差异的区分能力，两者从不同侧面刻画主成分的重要性。将二者相乘并归一化，即得兼顾信息量与区分度的组合权重

$$w_j = \frac{\alpha_j \beta_j}{\sum_{k=1}^{p} \alpha_k \beta_k}$$

式中j 取值为 $1, \ldots, p$。因为 α_j 与 β_j 均非负，且分母

$$\sum_{k=1}^{p} \alpha_k \beta_k > 0$$

所以各组合权重非负，且组合权重之和为 1，即

$$w_j \geq 0$$

$$\sum_{j=1}^{p} w_j = 1$$

即权重向量 w 构成主成分空间上的一个概率分布，其分量为

$$w = (w_1, w_2, \cdots, w_q).$$

TOPSIS 评价

在熵权基础上进行 TOPSIS 评价，先作向量规范化

$$u_{ij} = \frac{f_{ij}}{\sqrt{\sum_{i=1}^{n} f_{ij}^2}}$$

以消除量纲与正负号的影响并保持各样本之间的相对比例，再加权得

$$z_{ij} = w_j u_{ij}$$

记加权规范化矩阵为

$$Z = \begin{pmatrix} z_{11} & z_{12} & \cdots & z_{1p} \\ z_{21} & z_{22} & \cdots & z_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ z_{n1} & z_{n2} & \cdots & z_{np} \end{pmatrix}$$

取第 j 个主成分的正理想解与负理想解分别为

$$z_j^+ = \max_i z_{ij}$$

$$z_j^- = \min_i z_{ij}$$

则样本 i 到正理想解与负理想解的欧氏距离分别为

$$S_i^+ = \sqrt{\sum_{j=1}^{p} (z_{ij} - z_j^+)^2}$$

$$S_i^- = \sqrt{\sum_{j=1}^{p} (z_{ij} - z_j^-)^2}$$

相对贴近度定义为

$$C_i = \frac{S_i^-}{S_i^+ + S_i^-}$$

式中i取 1 至n。当权重向量非零且各主成分不全为常数时，两个距离不可能同时为零，故相对贴近度严格介于 0 与 1 之间，即

$$0 < C_i < 1$$

相对贴近度越大，表明样本越靠近正理想解、越远离负理想解，样本越优，按相对贴近度由大到小排序即得样本的优劣次序。为使评价结论以期望值的形式给出，进一步对第 j 个主成分定义样本i的相对优度

$$r_{ij} = \frac{z_{ij} - z_j^-}{z_j^+ - z_j^-}$$

它把加权取值线性压缩到 0 与 1 之间，其中取值为 0 表示该样本在此主成分上恰为最差，取值为 1 表示恰为最优。若第j个主成分的正理想解与负理想解重合，说明该主成分在所有样本上取值一致，此时其熵权为零，相应地组合权重亦为零，该主成分对评价的贡献恒等于零，于是约定这一情形下相对优度取 1，与组合权重为零的效果一致。由于各组合权重非负且全体组合权重之和为 1，可以把 w_{j} 理解为样本i在第j个主成分上取得相对优度r_{ij}这一事件的概率，于是样本i的综合评价值就是一个取值为r_{ij} 至r_{ip}的离散型随机变量ξ_i，其分布律为

$$P(\xi_i = r_{ij}) = w_j.$$

Q值

按离散型随机变量期望值的定义，即各取值与其对应概率乘积之和，样本 i 的评分期望值为

$$Q_i = E(\xi_i) = \sum_{j=1}^{p} w_j r_{ij}$$

这就是数据评分的Q值表达式。由相对优度取值介于 0 与 1 之间以及组合权重的归一性，立即得到Q值的取值范围

$$0 \le Q_i \le 1$$

且Q值随各主成分取值的增大而增大，即

$$\frac{\partial Q_i}{\partial z_{ij}} = \frac{w_j}{z_j^+ - z_j^-} \ge 0$$

再把相对贴近度对z_{ij}求偏导，由以下两式

$$\frac{\partial S_i^+}{\partial z_{ij}} = \frac{z_{ij} - z_j^+}{S_i^+} < 0$$

$$\frac{\partial S_i^-}{\partial z_{ij}} = \frac{z_{ij} - z_j^-}{S_i^-} > 0$$

可得

$$\frac{\partial C_i}{\partial z_{ij}} = \frac{S_i^+(z_{ij} - z_j^-)/S_i^- + S_i^-(z_j^+ - z_{ij})/S_i^+}{(S_i^+ + S_i^-)^2} > 0$$

即相对贴近度与期望值Q关于各主成分取值同为严格单调递增，两者的评价方向完全一致。特别地，当只保留单个主成分时，两个距离分别退化为

$$S_i^+ = z_1^+ - z_{i1}$$
$$S_i^- = z_{i1} - z_1^-$$

于是

$$Q_i = r_{i1} = \frac{z_{i1} - z_1^-}{z_1^+ - z_1^-} = \frac{S_i^-}{S_i^+ + S_i^-} = C_i$$

期望值Q与相对贴近度严格相等。由此可见，Q值是以数学期望的形式把熵权法、主成分分析的信息贡献与 TOPSIS 的理想解思想统一起来的综合评价指标，Q_i越大表示样本i的质量表现越优。领域级 Q 的计算则提取样本后取平均值

$$Q_{domain} = \frac{1}{n}\sum_{i}^{n} Q_i$$

结果

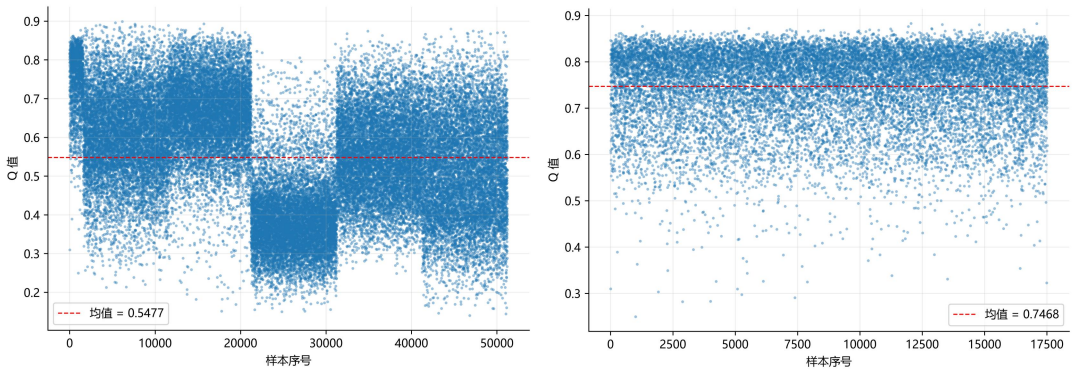
经计算，A1A2A3 的全部样本级Q值已保存在附件 A1_quality_scores.csv，A2_quality_scores.csv，A3_quality_scores.csv，正文给出它们的一些统计指标：平均值、区间、方差、领域级Q、散点图（图）。

表 2 样本级Q的统计

样本集	平均	区间	方差
A1	0.54771	[0.14024, 0.89919]	0.021061
A2	0.746821	[0.24962, 0.88291]	0.006541
A3	0.38944	[0.08838, 0.86380]	0.009118

表 3 领域级Q的统计

序号	领域	平均值-领域级Q
1	A1-ArXiv	0.602839
2	A1- Book	0.390437
3	A1- C4	0.550531
4	A1- CommonCrawl	0.510069
5	A1- GitHub	0.657095
6	A1- StackExchange	0.746247
7	A1- Wikipedia	0.743366
8	A2- ArXiv	0.746821
9	A3- GitHub	0.38944



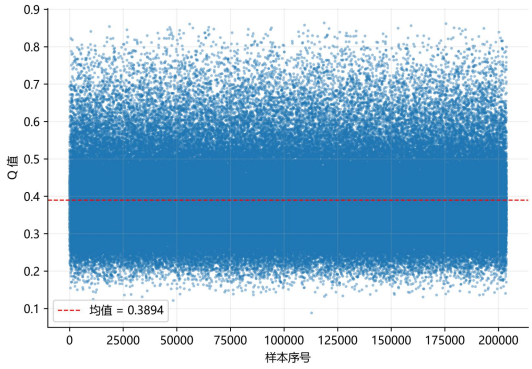


图 3 数据集 A1A2A3 的样本级Q散点图，A1 可见明显的领域分化

3.3 冲突指标对和冲突分析

本小节将先人工选取冲突指标对，并定义冲突率和冲突强度完成评分Q的修正。需要特别说明的是，冲突指标对无关领域指标分配，因为其常常在分配中视作同向正性或同向负性。

（i）冲突定义：全部冲突对集合 $P(j,k)$，其中每一对指标的预期同向理由与判断力度如下表所示。

表 4 冲突指标对和原因

序号	指标对	原因
1	fineweb_edu → Qurater[Educational Value]	都直接评价教育价值
2	Qurater[Required Expertise] → modernbert_professionalism	都涉及理解文本所需的专业程度但“专业”不等于“优质”
3	fluency_en → Qurater[Writing Style]	流畅性通常是良好写作的一部分
4	fluency_en → modernbert_readability	流畅文本通常较易阅读
5	Qurater[Writing Style] → modernbert_readability	表达清晰通常有助于可读性
6	modernbert_cleanliness → Qurater[Writing Style]	排版、文本整洁与写作观感通常同向
7	rps_doc_frac_chars_top_2gram → rps_doc_frac_chars_top_3gram	两者都刻画局部重复；反向转换后通常同向
8	rps_doc_frac_unique_words → rps_doc_unigram_entropy	都反映词汇分布的多样性
9	反向后的 top_2gram → frac_unique_words；反向后的 top_3gram → unigram_entropy	重复较少的文本通常更具词汇多样性

10	lines_ending_with_terminal_punctuation_mark → fluency_en	完整叙述句通常更符合流畅自然语言

第 i 个样本上的指标对 (j,k) 是否发生冲突，用指示变量 $C_{i,jk}$ 表示，其取值由指示函数给出，即

$$C_{i,jk} = 1\{\max(u_{ij},u_{ik}) \geq 0.8, \min(u_{ij},u_{ik}) \leq 0.2\}$$

（ii）冲突强度定义为

$$S_{i,jk} = C_{i,jk}\frac{|u_{ij}-u_{ik}| - 0.6}{0.4}, 0 \leq S_{i,jk} \leq 1$$

构造式子反应了冲突在源数据评分中的相对强度，0.4 由 1-0.6 得到

（iii）冲突发生率定义为

$$\hat{p}_{jk} = \frac{\sum_i C_{i,jk}}{n_{jk}}$$

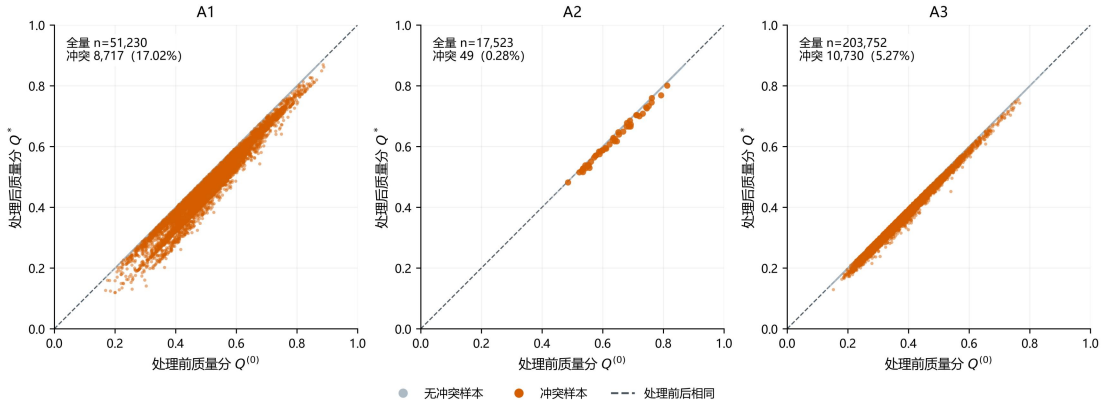
（iv）修正后的Q值为

$$Q_i^* = Q_i^{(0)} - \lambda \sum_{(j,k)\in\mathcal{P}} C_{i,jk}\, S_{i,jk}[1-(1-\alpha)p_{jk}]$$

其中参数为 λ 和 α，需保证让 Q_i 保持在 (0,1) 之间。经计算，

$$\begin{cases}\lambda = 0.5\\ \alpha = 0.5\end{cases}$$

冲突的计算结果如图 表所示。A1 的整体冲突率为 17.02%，8717 个；A2 的冲突率 0.28%，49 个；A3 为 5.27%，10730 个。而 A1 样本集中出现多对冲突的现象。在冲突发生后，其样本Q值均下降，图 4 反应了偏离斜率为 1 的直线$y = x$的程度。样本Q值的下降导致了整体均值的小幅下降。



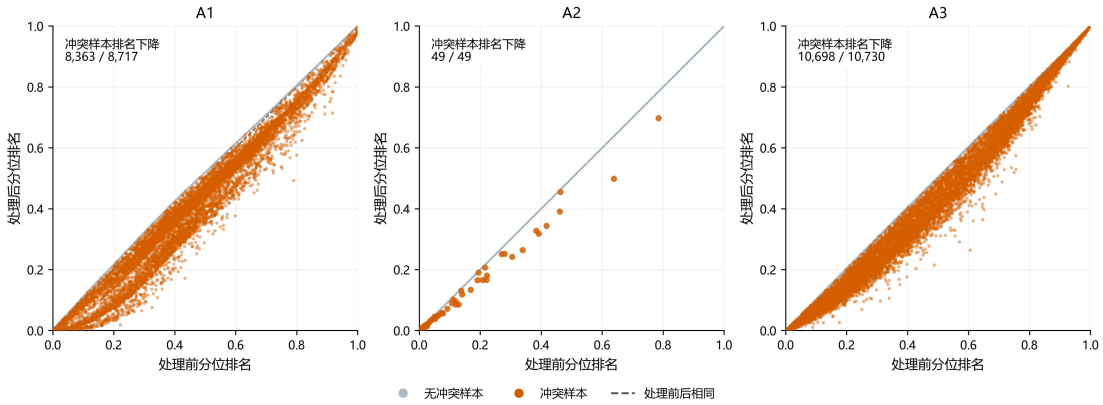


图 4 三个样本集的冲突下降数量和下降率

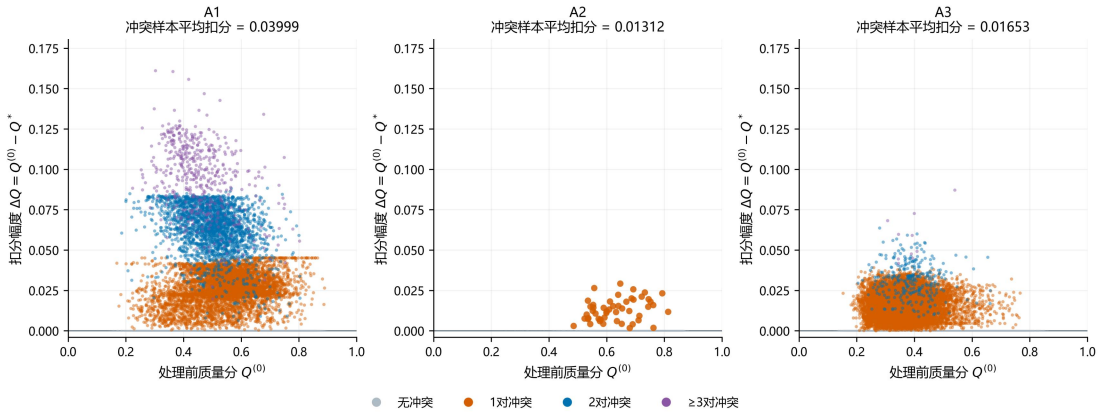


图 5 按冲突对数分类和分值变化情况

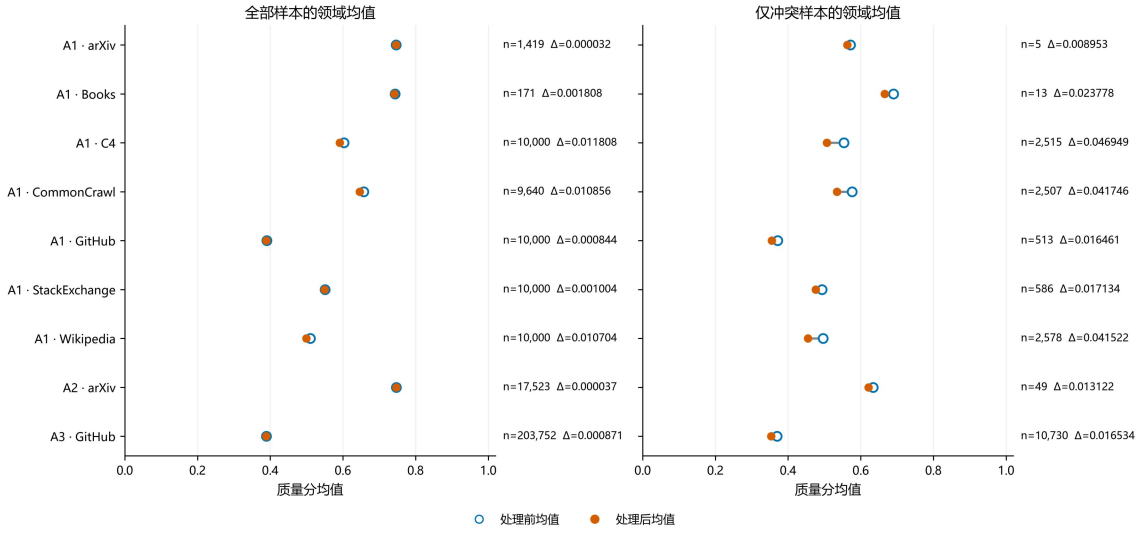


图 6 全部样本按领域Q均值及其变化；全部冲突样本按领域Q均值及其变化

表 5 数据集样本级Q的冲突统计

样本集	冲突率	冲突量

A1	17.02%	8717
A2	0.28%	49
A3	5.27%	10730

表 6 领域Q的冲突统计

序号	领域	Q均值	修正后Q均值	变化
1	A1-ArXiv	0.602839	0.591032	0.011808
2	A1- Book	0.390437	0.389592	0.000844
3	A1- C4	0.550531	0.549527	0.001004
4	A1- CommonCrawl	0.510069	0.499364	0.010704
5	A1- GitHub	0.657095	0.646239	0.010856
6	A1- StackExchange	0.746247	0.746216	0.000032
7	A1- Wikipedia	0.743366	0.741559	0.001808
8	A2- ArXiv	0.746821	0.746784	0.000037
9	A3- GitHub	0.38944	0.388569	0.000871

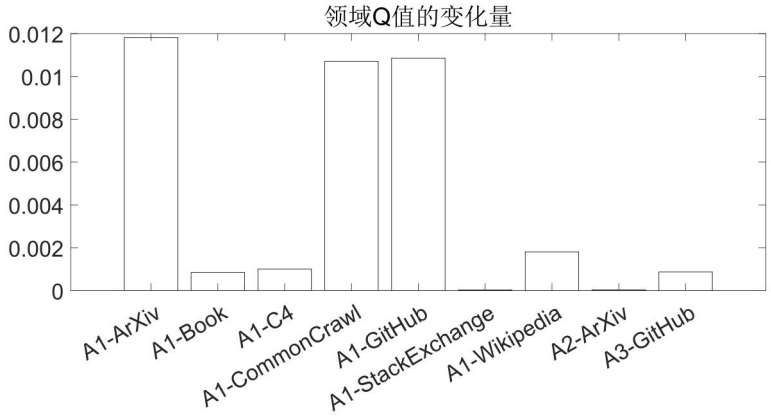


图 7 领域Q值在同一坐标中的变化量

3.4 对数 L-p模型和最优p

本问题要求建立 Loss 和配比向量$\boldsymbol{p}$的定量关系，训练集为 A4A5，外推集为 A6-A11，讨论稳健性。首先，观察并分析 A4A5 的数据发现如下基本特点：

（i）Loss 的计算指明了领域；

（ii）对于某一特定领域，Loss 随该领域的配比增大而减少；

（iii）配比数据为稀疏的，说明p_{ik}为 0 时 Loss 不能为 0；

（iv）配比覆盖全部 17 个领域，Loss 则只有 13；
（v）Loss 的值大约在区间[3,7]之间。

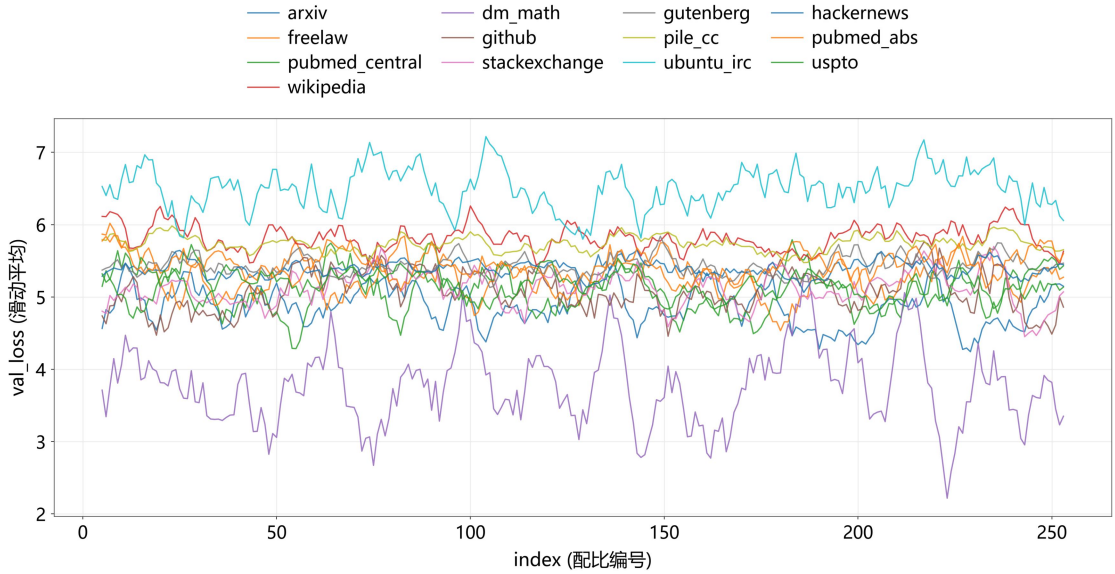


图 8 训练集 Loss 的总体分布

将问题数学化，就是在 Loss 和$\boldsymbol{p}$之间寻找具体映射，即寻找

$$L = A + f(\boldsymbol{p})$$

中f的具体形式，A是 Loss 的某一上下界。由于数据以矩阵形式给出，且 Loss 按领域计算，我们自然想到矩阵乘法

$$L_{ij} = A_j + w_{17\times13}P_{i:}$$

这里，j对应 Loss 中的不同领域，i对应不同样本。但该多维线性模型显然为一超定方程组，如构成解空间则为一超平面，较为刚性。因此考虑对数模型

$$L_{ij} = A_j - \sum_{k=1}^{17} w_k \ln(p_{ik} + 1)$$

可以同时满足所分析的五个特点，只需获得 13 个A_j和 17 个w_k，尽可能降低了参数量。同时，若作变换

$$x_k = ln(p_{ik} + 1)$$

则 Loss 的表达式又可以看做是线性模型

$$L_{ij} = A_j - \sum_{k=1}^{17} w_k x_k = A_j - wX$$

这就回到了矩阵或向量乘法，故认为对数模型较为合理。

经分析，我们设置了两种对数模型

M1:

$$L_{ij} = A_j - \sum_{k=1}^{17} w_k \ln(p_{ik} + 1)$$

M2：

$$L_{ij} = A_j + \sum_{k}^{17} b_{kj} \ln\left(1+\frac{p(i,k)}{\tau}\right)$$

非线性问题常常使用最小二乘法求解。由于两个模型关于参数都是对数线性的，引入特征 $\varphi_{ik} = \ln(1+p_{ik})$ 与 $\psi_{ik} = \ln(1 + p_{ik}/\tau)$，则

$$\text{M1: } L_{ij} = A_j - \sum_{k=1}^{17} w_k \varphi_{ik},$$

$$\text{M2: } L_{ij} = A_j + \sum_{k=1}^{17} b_{kj} \psi_{ik}.$$

因此 M1 是 30 个参数、M2 是 234 个参数的线性最小二乘问题。特别地，M2 中第 j 个目标的方程只含 A_j 与 $b_{1j},\ldots,b_{17j}$，可分解为 13 个相互独立的 18 维线性最小二乘；而 M1 的 w_k 在所有 j 上共享，必须联合估计。

把全部 (i,j) 方程拉直为 $\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \mathbf{r}$，标准最小二乘最小化残差平方和

$$J(\boldsymbol{\theta}) = \sum_{i=1}^{256}\sum_{j=1}^{13} \left(L_{ij}-\hat{L}_{ij}\right)^2 = \|\mathbf{y}-\mathbf{X}\boldsymbol{\theta}\|_2^2$$

其中 M1 的参数 $\boldsymbol{\theta} = (A_1,\ldots,A_{13},w_1,\ldots,w_{17})^{\mathsf{T}} \in \mathbb{R}^{30}$，设计矩阵 $\mathbf{X} \in \mathbb{R}^{3328\times 30}$ 的第 (i,j) 行在 A_j 位置为 1、在 w_k 位置为 $-\varphi_{ik}$；M2 逐个 j 求解，$\boldsymbol{\theta}_j = \left(A_j,b_{1j},\ldots,b_{17j}\right)^{\mathsf{T}} \in \mathbb{R}^{18}$，设计矩阵 $\mathbf{X}_j \in \mathbb{R}^{256\times 18}$ 的第 i 行为 $(1,\psi_{i1},\ldots,\psi_{i17})$。最优解满足正规方程

$$\mathbf{X}^{\mathsf{T}}\mathbf{X}\widehat{\boldsymbol{\theta}} = \mathbf{X}^{\mathsf{T}}\mathbf{y}$$

当 $\mathbf{X}$ 列满秩时，闭式解为

$$\widehat{\boldsymbol{\theta}} = (\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathsf{T}}\mathbf{y}$$

若 $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ 接近奇异（特征强相关或样本不足），需改用伪逆或岭正则化

$$\widehat{\boldsymbol{\theta}} = (\mathbf{X}^{\mathsf{T}}\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^{\mathsf{T}}\mathbf{y}$$

标准最小二乘的梯度下降形式如下。目标 $J(\boldsymbol{\theta}) = \frac{1}{2}\|\mathbf{y}-\mathbf{X}\boldsymbol{\theta}\|_2^2$ 的梯度为

$$\nabla J(\boldsymbol{\theta}) = -\mathbf{X}^{\mathsf{T}}(\mathbf{y}-\mathbf{X}\boldsymbol{\theta}) = \mathbf{X}^{\mathsf{T}}(\mathbf{X}\boldsymbol{\theta}-\mathbf{y})$$

梯度下降按下式迭代，$\eta > 0$ 为步长

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\mathbf{X}^{\mathsf{T}}\left(\mathbf{X}\boldsymbol{\theta}^{(t)}-\mathbf{y}\right)$$

为保证收敛，步长需满足

$$0 < \eta < \frac{2}{\lambda_{max}(\mathbf{X}^{\mathsf{T}}\mathbf{X})}$$

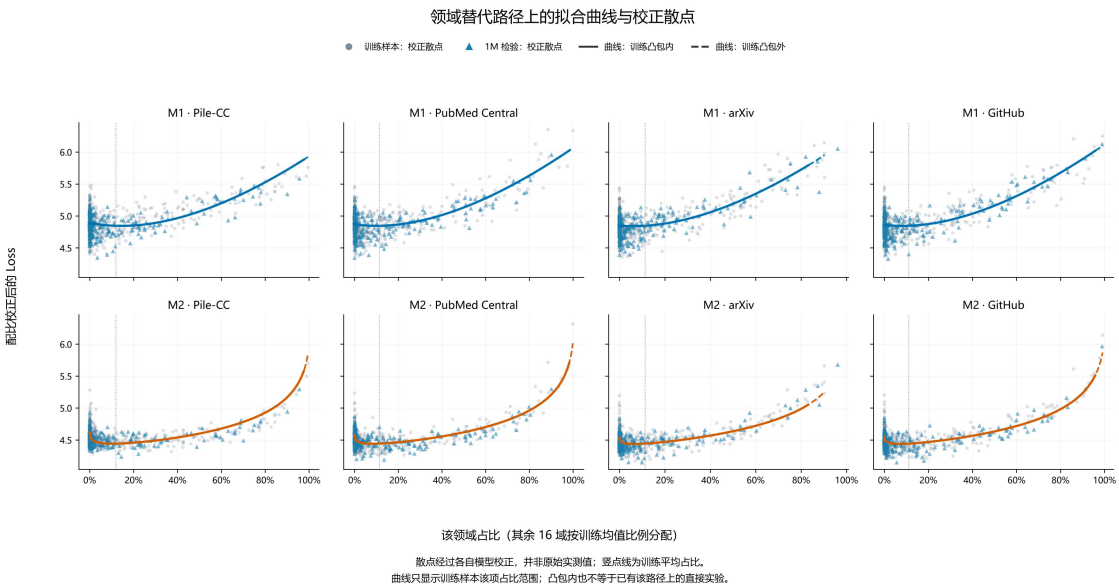
其中 λ_{max} 表示矩阵的最大特征值。记 $r_{ij} = L_{ij} - \hat{L}_{ij}$ 为当前残差。两种模型对截距的更新相同

$$A_j^{(t+1)} = A_j^{(t)} + \eta\sum_{i=1}^{256} r_{ij}^{(t)}$$

M1 对共享系数的更新与 M2 对系数的更新分别为

$$w_k^{(t+1)} = w_k^{(t)} - \eta \sum_{i=1}^{256} \sum_{j=1}^{13} r_{ij}^{(t)} \varphi_{ik}$$

$$b_{kj}^{(t+1)} = b_{kj}^{(t)} + \eta \sum_{i=1}^{256} r_{ij}^{(t)} \psi_{ik}.$$



13 个验证域的独立检验：A6/A7，1M

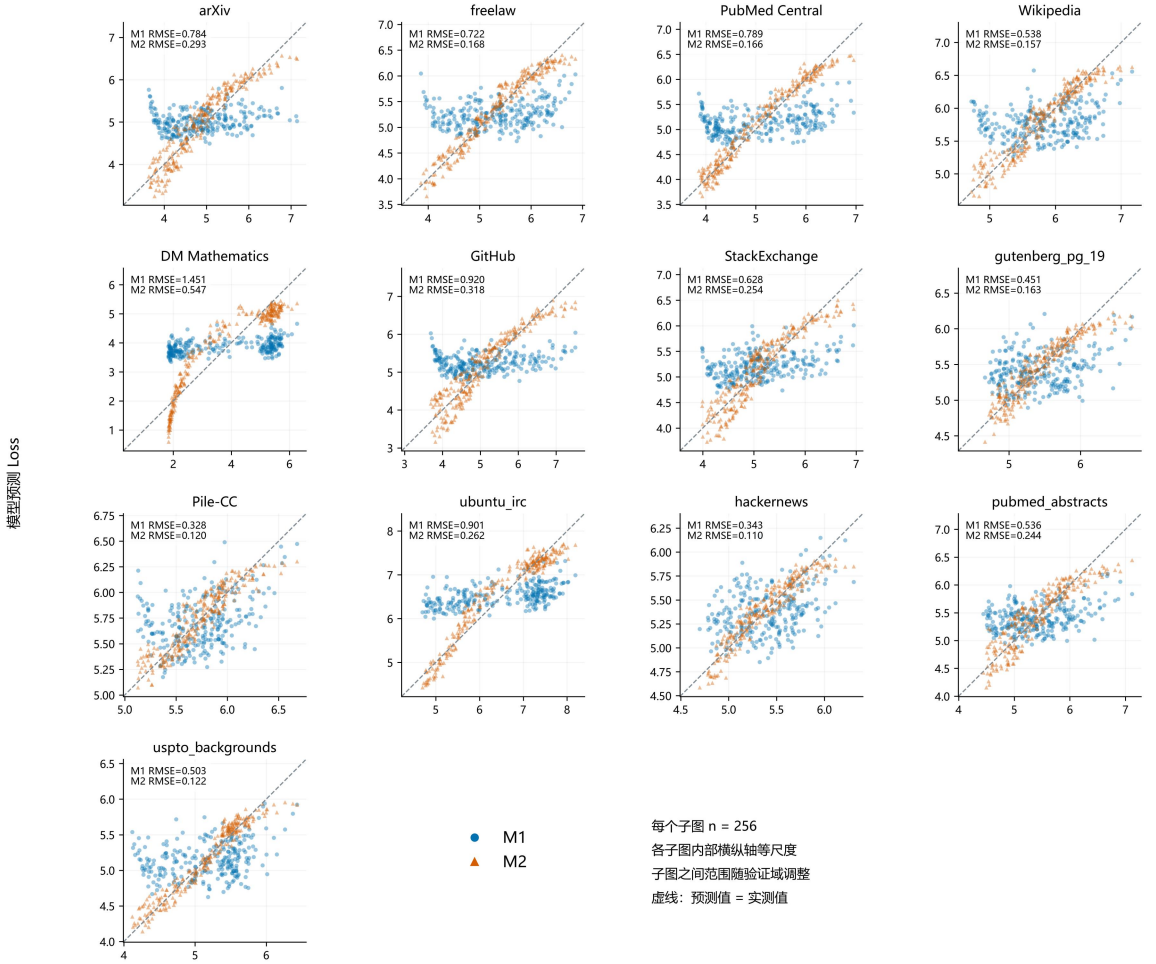


图 9 M1M2 在各个领域的性能表现

经过计算，M1 的表现要更好，即定量关系为

$$L_{ij} = A_j - \sum_{k=1}^{17} w_k \ln(p_{ik} + 1)$$

其中分领域的参数集合A, w分别为

$$A = \begin{pmatrix} 9.06695 \\ 9.30546 \\ 9.19881 \\ 9.83378 \\ 7.91646 \\ 9.30240 \\ 9.26873 \\ 9.46865 \\ 9.75068 \\ 10.5764 \\ 9.42660 \\ 9.20069 \end{pmatrix}, \qquad w^T = \begin{pmatrix} 4.59906 \\ 4.55764 \\ 4.63337 \\ 4.76584 \\ 4.59826 \\ 5.02057 \\ 4.68530 \\ 5.47722 \\ 5.06743 \\ 7.56486 \\ 4.33547 \\ 4.95327 \\ 5.71521 \\ 4.25841 \\ 4.89184 \\ 4.47159 \\ 4.43938 \end{pmatrix}$$

结果已保存在附件 problem1_3_solution.csv 中。

4. 问题二：广义标度率$L(N, D, Q, p)$的显示表达

4.1 指数项Q和分配项p

本问题和下一问题的显式表达均使用的是由 DeepMind 提出的 Chinchilla-Scaling Law[2]，其显示表达常为

$$L(N, D, C) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} + \frac{C}{\tau}$$

式中τ为混合语料比例，也常用r表示。

附件	行数	数据性质	N	D	Q	Loss	17域p
B1	1,176	来源训练日志	有	有	—	有	—
B2	1,029	半合成	有	有	—	有	—
B3	4,000	插值轨迹	有	有	—	有	—
B4	57	资料整理	有	有	—	有	—
B5	44	文献整理	有	有	—	有	—
B6	360	半合成	有	有	有	有	—
B7	450	半合成	有	有	有	有	—
B8	1,704	半合成 / 外推	有	有	有	有	—
B9	132	模型元数据	有	部分	—	—	—
B10	128	估算 Loss	有	有	—	有	—
B11	18	模型族索引	—	—	—	—	—
B12	1,386	检查点索引	—	—	—	—	—

图 10 问题二数据集情况一览

4.2 岭回归模型和求解

普通最小二乘（OLS）通过最小化残差平方和估计回归系数。当自变量之间存在较强多重共线性，或样本量相对特征数不足时，设计矩阵的列近似线性相关，$\mathbf{X}^{\mathsf{T}}\mathbf{X}$接近奇异，OLS 估计量的方差很大、数值上不稳定。岭回归在残差平方和之上加入系数向量的 L2 范数惩罚，以引入微小偏差为代价，换取估计方差的大幅下降，从而得到更稳定、更可解释的系数。

设标准化后的设计矩阵为 $\mathbf{X} \in \mathbb{R}^{n \times p}$（$n$ 个样本、p 个特征），响应向量为 $\mathbf{y} \in \mathbb{R}^{n}$（已中心化），待估系数向量为 $\boldsymbol{\beta} \in \mathbb{R}^{p}$。OLS 的目标函数为

$$min_{\boldsymbol{\beta}}\|\mathbf{y}-\mathbf{X}\boldsymbol{\beta}\|_2^2.$$

岭回归在残差平方和上增加系数平方和的惩罚，目标函数为

$$\widehat{\boldsymbol{\beta}}_\lambda = \mathrm{argmin}_{\boldsymbol{\beta}}\{\|\mathbf{y}-\mathbf{X}\boldsymbol{\beta}\|_2^2+\lambda\|\boldsymbol{\beta}\|_2^2\}.$$

其中 $\lambda \geq 0$ 为岭参数（正则化强度）。$\lambda = 0$ 时退化为 OLS；λ 越大，系数向原点收缩越强，但偏差也越大。

对目标函数关于 $\boldsymbol{\beta}$ 求梯度并令其为零，得到正规方程

$$(\mathbf{X}^\mathrm{T}\mathbf{X}+\lambda\mathbf{I})\widehat{\boldsymbol{\beta}}_\lambda = \mathbf{X}^\mathrm{T}\mathbf{y}.$$

当 $\lambda > 0$ 时矩阵 $\mathbf{X}^\mathrm{T}\mathbf{X} + \lambda\mathbf{I}$ 正定，闭式解为

$$\widehat{\boldsymbol{\beta}}_\lambda = (\mathbf{X}^\mathrm{T}\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^\mathrm{T}\mathbf{y}.$$

对 $\mathbf{X}$ 作奇异值分解 $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\mathrm{T}$，其中 $\boldsymbol{\Sigma}$ 的对角元为奇异值 $d_1 \geq d_2 \geq \cdots \geq d_p \geq 0$。岭估计可展开为

$$\widehat{\boldsymbol{\beta}}_\lambda = \sum_{j=1}^{p} \frac{d_j}{d_j^2+\lambda}\mathbf{u}_j^\mathrm{T}\mathbf{y}\mathbf{v}_j.$$

对比 OLS 的 $\widehat{\boldsymbol{\beta}} = \sum_{j=1}^{p} d_j^{-1}\,\mathbf{u}_j^\mathrm{T}\mathbf{y}\mathbf{v}_j$，岭回归对每个奇异方向乘了一个收缩因子

$$s_j = \frac{d_j^2}{d_j^2+\lambda}.$$

奇异值越小的方向收缩越强，这正是岭回归抑制共线性方向、降低方差的机制。

岭回归的惩罚项对系数尺度敏感，拟合前必须对特征做标准化。对第 j 个特征的第 i 个观测：

$$z_{ij} = \frac{x_{ij}-\overline{x}_j}{s_j}.$$

其中 $\overline{x}_j$ 与 s_j 分别为第 j 个特征的均值和（样本）标准差；响应中心化为 $\mathbf{y}-\overline{y}\mathbf{1}$。预测时用训练阶段记录的均值、标准差对新样本做相同变换，最后把预测值加回 $\overline{y}$。

岭参数的选择直接决定偏差—方差折中。常用方法有三种。

（1）K 折交叉验证。 将样本划分为 K 折，对候选 λ 逐次留一折作验证、其余折拟合，取平均验证误差最小者。常采用“最小误差加一个标准误”规则，在阈值内选更小的 λ，得到更简约的模型。

（2）广义交叉验证（GCV）。 用留一交叉验证的解析近似，计算

$$\mathrm{GCV}(\lambda) = \frac{1}{n}\cdot\frac{\left\|\boldsymbol{y}-\boldsymbol{X}\widehat{\boldsymbol{\beta}}_\lambda\right\|_2^2}{[1-tr(\boldsymbol{H}_\lambda)/n]^2},$$

其中 $\boldsymbol{H}_\lambda = \boldsymbol{X}(\boldsymbol{X}^T\boldsymbol{X}+\lambda\boldsymbol{I})^{-1}\boldsymbol{X}^T$ 为帽子矩阵。GCV 免去反复折分，计算量小。

（3）岭迹图。 绘制各系数随 λ 变化的曲线，选取系数开始趋于稳定的最小 λ。

将上述内容整理为可复现的步骤：

1. 读入数据，构造设计矩阵 $\mathbf{X}$ 与响应向量 $\mathbf{y}$；
2. 中心化与标准化，记录每列的均值与标准差；
3. 设定候选岭参数网格 $\lambda \in \{\lambda_1, ..., \lambda_m\}$；
4. 对每个 λ 求解 $\widehat{\boldsymbol{\beta}}_\lambda = (\mathbf{X}^\mathrm{T}\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^\mathrm{T}\mathbf{y}$；
5. 用 K 折交叉验证评估每个 λ，记录平均验证误差；
6. 选择最优 λ^*（最小误差，或最小误差加一个标准误）；
7. 用 λ^* 在全部训练数据上重新拟合，得到最终系数；
8. 反变换回原始尺度，输出预测值与评估指标（RMSE、R^2）。

记 $\hat{y}_i$ 为预测值，$\overline{y}$ 为响应均值。均方根误差与决定系数定义为

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}, R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{\sum_{i=1}^{n}(y_i-\overline{y})^2}.$$

RMSE 越小、R^2 越接近 1 表示拟合越好；R^2 可为负，表示甚至不如响应均值基准。系数轨迹与预测残差图用于检查收缩是否合理、是否存在离群点。

岭回归与 LASSO、弹性网同属线性模型的正则化方法，差异在于惩罚项的形式。

表 7 常用回归算法对比

方法	惩罚项	主要特点
OLS	无	无偏；共线性下不稳定
岭回归	$\lambda\|\|\boldsymbol{\beta}\|\|_2^2$	收缩但不稀疏，稳定
LASSO	$\lambda\|\|\boldsymbol{\beta}\|\|_1$	可稀疏化，兼具变量选择
弹性网	$\lambda_1\|\|\boldsymbol{\beta}\|\|_1 + \lambda_2\|\|\boldsymbol{\beta}\|\|_2^2$	兼顾收缩与稀疏

当需要稳定性、特征之间相关性较强时选岭回归；当需要变量选择时选 LASSO；两者折中选弹性网。本问题使用岭回归求解，其策略见图 11

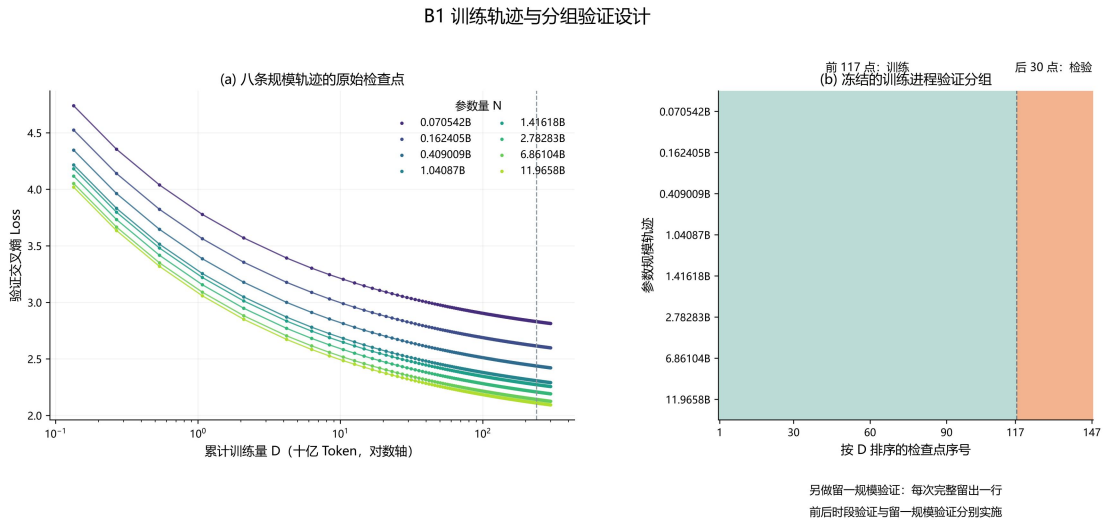


图 11 分组验证和训练轨迹

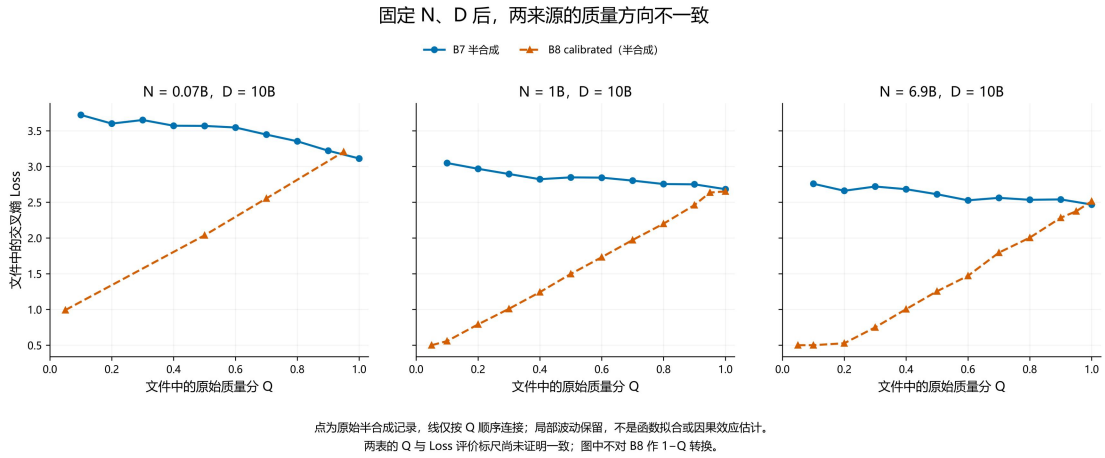


图 12 固定N, D的性能

得到最优广义标度率表达式和参数取值，式中p_0是上一问题中的最优配比

$$L(D,N,Q,\boldsymbol{p}) = E + AN^{-\alpha} + BD^{-\beta} + \eta_Q(1-Q^{\kappa}) + \lambda_{\text{mix}}\sum_{k=1}^{17} w_k \ln\left(\frac{1+p_k}{1+p_{0k}}\right)$$

表 8

参数	估计值
E	1.689798
a	0.35398
b	1.240306
alpha	0.339977
beta	0.279878

5. 问题三：算力约束下的最优 Loss 配置

5.1 多目标约束问题

问题三在给定总算力预算C和上下文长度 L_{ctx}的条件下，联合决定模型规模、训练数据量、数据质量和数据配比，使预测交叉熵损失最小。决策变量为

$$(N, D, Q, \boldsymbol{p})$$

其中 N为模型参数量，D为训练 token 数，Q为数据质量，而

$$\boldsymbol{p} = (p_1, \ldots, p_{17})$$

p_k为第k个数据域的 token 配比。

采用问题二结果的形式，并进行单位换算，则最优化问题为：

$$\text{argmin}\, L = E + \mathrm{A}N_1^{-\alpha} + \mathrm{B}D_1^{-\beta} + \eta_Q(1 - Q^{\kappa}) + \lambda_{\text{mix}} \sum_{k=1}^{17} w_k \ln\left(\frac{1 + p_k}{1 + p_{0k}}\right),$$

$$s.t. \begin{cases} 10^{18}(6 + \eta_{attn} L_{ctx}) N_1 D_1 + 10^9 D_1 [g(Q) - g(Q_0)]_+ \le C, \\ E = 1.590644124, \\ A = 0.422869586, \\ B = 1.317513923, \\ \alpha = 0.339976582, \\ \beta = 0.279878129, \\ \eta_Q = 0.361995286, \\ \kappa = 1, \\ \lambda_{mix} = 1, \\ [x]_+ = max(x, 0), \\ \eta_{attn} = 2 \times 10^{-4}, \\ Q_0 = 0.6575, \\ L_{ctx} = 2048, \\ w_k \in w. \end{cases}$$

其中单位换算的变量代换为

$$N_1 = \frac{N}{10^9}$$

$$D_1 = \frac{D}{10^9}$$

集合w为前面问题所算出的最优系数。

5.2 粒子群算法和求解

粒子群优化（Particle Swarm Optimization，PSO）是一种基于群体智能的随机优化算法，受鸟群觅食行为启发。每个粒子代表解空间中的一个候选解，通过个体历史最优位置与群体历史最优位置共同引导搜索。与遗传算法相比，PSO 结构简单、参数少、收敛快，广泛用于连续优化问题。

设搜索空间为 D 维，粒子 i 的位置记为 $\mathbf{x}_i = (x_{i1}, \ldots, x_{iD})^{\mathsf{T}}$，表示候选解；速度 $\mathbf{v}_i$ 表示移动方向与步长。粒子 i 的历史最优位置记为 $\boldsymbol{p}_i$，群体历史最优位置记为 $\boldsymbol{g}$。

第 $t+1$ 次迭代，速度与位置按下式更新：

$$\mathbf{v}_i^{(t+1)} = \omega \mathbf{v}_i^{(t)} + c_1 r_1 \left(\mathbf{p}_i - \mathbf{x}_i^{(t)}\right) + c_2 r_2 \left(\mathbf{g} - \mathbf{x}_i^{(t)}\right),$$

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)}.$$

其中 ω 为惯性权重，控制对上一代速度的继承；c_1, c_2 为学习因子，分别控制向个体最优与全局最优学习的强度；r_1, r_2 为 $[0,1]$ 上均匀分布的随机数。常用设定 $c_1 = c_2 = 1.7$ 至 2.0，惯性权重由 0.9 线性递减到 0.4。

Algorithm

1	初始化：随机生成 N 个粒子的位置 $\mathbf{x}_i$ 与速度 $\mathbf{v}_i$，计算各粒子适应度
2	记录个体最优 $\mathbf{p}_i = \mathbf{x}_i$，群体最优 $\mathbf{g} = \mathrm{argmin}_i f(\mathbf{p}_i)$
3	按更新公式更新每个粒子的速度 $\mathbf{v}_i$ 与位置 $\mathbf{x}_i$
4	越界处理：将位置与速度约束到可行范围
5	计算新位置的适应度，更新个体最优 $\mathbf{p}_i$ 与群体最优 $\mathbf{g}$
6	判断终止条件：满足则输出 $\mathbf{g}$，否则返回步骤 3

解得最优方案配置各参数和 ***Loss*** 值如表 ，全部结果已保存在附件 problem1_3_solution.csv 中。

表 9 最优配置的主要参数取值

g_function	budget	Q0	lctx	***Loss***
exp	1.00E+19	0.6575	2048	2.083963
exp	1.00E+22	0.6575	2048	1.008086
exp	1.00E+24	0.6575	2048	0.744495
log	1.00E+19	0.6575	2048	2.037354
log	1.00E+22	0.6575	2048	1.002931
log	1.00E+24	0.6575	2048	0.745728
power	1.00E+19	0.6575	2048	2.083963
power	1.00E+22	0.6575	2048	1.009051
power	1.00E+24	0.6575	2048	0.744562
N_billion	D_billion	Q	budget_used	***Loss***
0.267471	5.832996	0.6575	1.00E+19	2.083963
6.636377	217.1362	1	1.00E+22	1.008086
49.03469	3146.531	1	1.00E+24	0.744495
0.368062	3.22039	1	1.00E+19	2.037354
6.183762	247.638	1	1.00E+22	1.002931
48.53963	3206.509	1	1.00E+24	0.745728
0.267471	5.832999	0.6575	1.00E+19	2.083963
6.719722	212.1508	1	1.00E+22	1.009051
49.13135	3135.014	1	1.00E+24	0.744562

6. 问题四：预测和 benchmark

6.1 问题四的模型

常见的十种大模型 benchmark：

MMLU 是覆盖 57 个学科的多项选择测试，包含约 1.6 万道题，涵盖 STEM、人文、社科、法律和医学等领域，主要考察模型的知识广度与基础推理能力。当前前沿模型得分普遍在 88%至 94%之间，逐渐趋于饱和，因此它更适合作为通用基线，而不宜单独用来判断模型强弱。

GSM8K 由约 8500 道小学数学应用题组成，每道题需要 2 到 8 步算术推理才能得出最终数值。它曾是衡量链式思考能力的入门标准，但如今顶尖模型几乎可以拿满分，因此在区分前沿模型方面已经不够灵敏，更多被用作基础数学能力的基线测试。

MATH 是一个竞赛数学题库，涵盖代数、几何、数论和概率等内容，难度从 AMC 级别延伸到奥数水平。其常用子集 MATH-500 包含 500 道精选题目，专门考察符号推理、公式运用和多步证明能力，比 GSM8K 更具挑战性，是衡量模型数学实力的常用指标。

HumanEval 包含 164 道 Python 函数生成题，每个问题都有隐藏的单元测试，通过 pass@k 指标评估模型能否生成正确的代码。HumanEval+版本进一步扩充了测试用例，降低了碰巧通过的概率。这个基准客观且可执行，但题库老旧且容易

被污染，前沿模型已经接近饱和，目前只适合作为基础代码生成的基线。

MT-Bench 设计了 80 组两轮对话，覆盖写作、推理、数学、代码和角色扮演等多种场景，由强模型按照 1 到 10 分进行评分。它重点考察多轮对话中的一致性、追问适应能力和指令服从程度，比单轮问答更贴近真实的客服或助手场景，缺点是样本量较少且裁判存在偏差。

Chatbot Arena 由 LMSYS 组织，采用用户盲选的方式让真实用户比较两个匿名模型的回答并投票，最终计算出 Elo 分数。它能够反映真实人群的偏好和开放域的体验感受，不受官方自报分数的干扰，但收敛速度较慢且偏向英文通用场景，在专业领域需要另外补充测试。

MMMU 是大学级别的多模态选择题库，包含大量理工科、医学、商业和人文学科的图表、公式和截图。它要求模型同时理解图像内容和专业知识并进行跨模态推理，是目前评估视觉语言模型学术能力的主要榜单之一，新版 MMMU-Pro 进一步提高了难度和抗猜测性。

TruthfulQA 包含 817 道容易引发错误回答的问题，涉及阴谋论、常识误区和医疗误解等方面。它的目的是检测模型是否倾向于说出真相而不是附和常见的错误先验，对于评估模型的对齐程度、幻觉风险和安全性非常关键，是客服和医疗问答场景的基础安全指标。

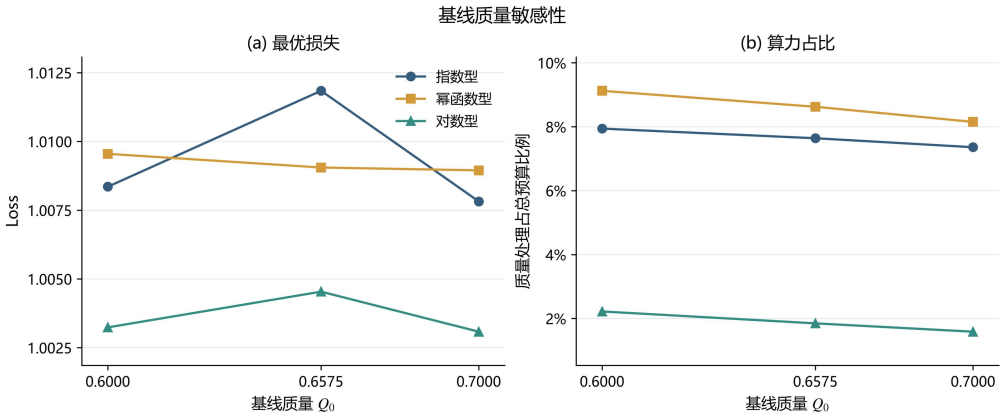
SWE-bench 通过提供真实的 GitHub Issue 和完整的代码仓库，要求模型生成补丁并通过 FAIL_TO_PASS 和 PASS_TO_PASS 两类测试来验证修复效果。Verified 子集包含 500 个人工核验的任务，重点考察模型阅读代码库、定位问题和修改多文件的能力，是衡量工程型智能体的核心基准。

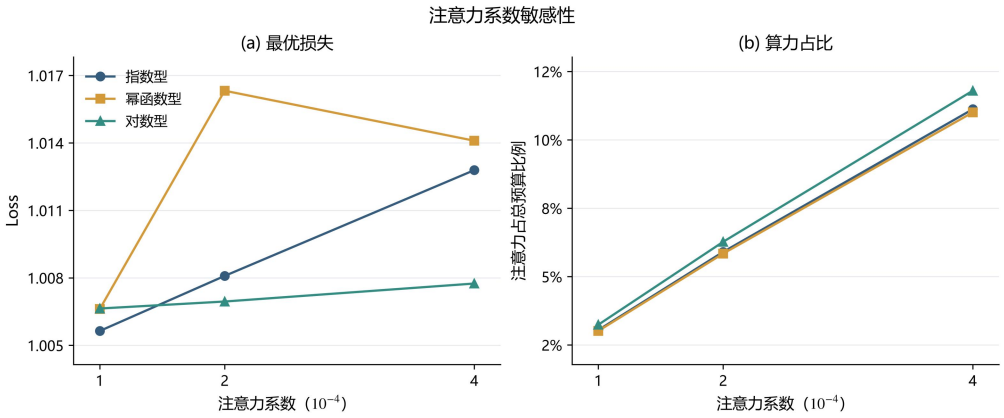
BIG-Bench 最初包含 200 多个社区提交的任务，覆盖算法、语言、常识和科学等多个维度，而 BBH 从中挑选出 23 个高难度的子集，如日期推理、逻辑演绎和因果判断等。这些任务刻意避免依赖简单的模式匹配，非常适合发现模型在复杂推理方面的短板，比单一学科的测试更加全面。

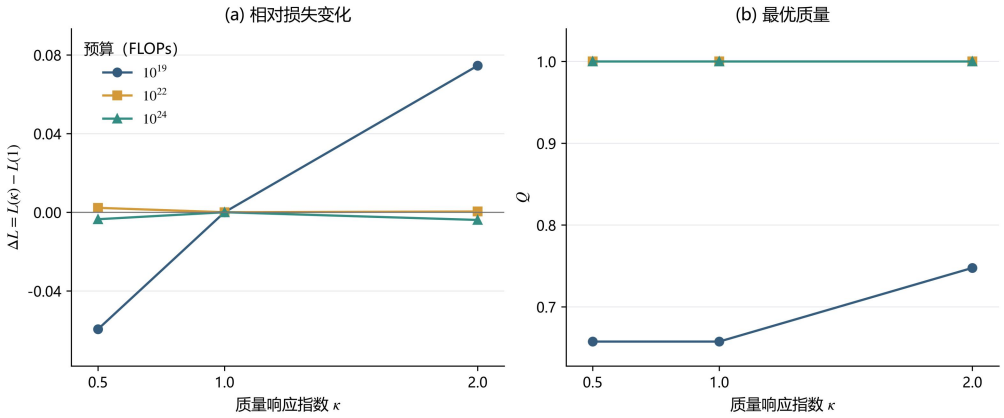
6.2 问题四的预测和结果

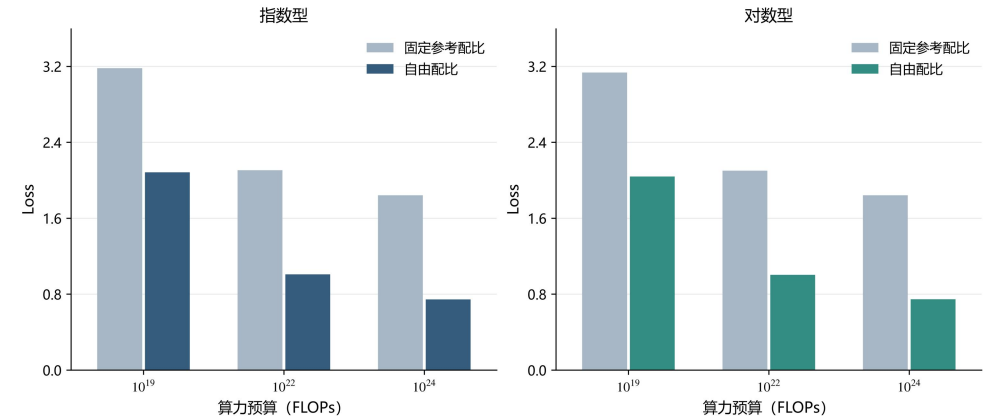
7. 模型评价

7.1 粒子群算法的灵敏度分析


基线质量敏感性
(a) 最优损失
(b) 算力占比
指数型
幂函数型
对数型
Loss
质量处理占总预算比例
基线质量 Q_0
基线质量 Q_0


注意力系数敏感性
(a) 最优损失
(b) 算力占比
指数型
幂函数型
对数型
Loss
注意力占总预算比例
注意力系数（10^{-4}）
注意力系数（10^{-4}）


(a) 相对损失变化
(b) 最优质量
预算（FLOPs）
10^{19}
10^{22}
10^{24}
$\Delta L = L(\kappa) - L(1)$
Q
质量响应指数 κ
质量响应指数 κ

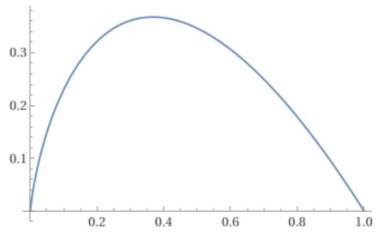

指数型
对数型
固定参考配比
自由配比
Loss
算力预算（FLOPs）
算力预算（FLOPs）

7.2 模型缺点

1.对 PCA 进行了完整的理论推导，但未实际使用；

2.第二问中尽管测试集的 RMSE 较低约 0.1，但验证集较高，达到 Loss 的 300%约 2~3；

3.信息熵函数$-xlnx$会在e^{-1}取到最大值，尚不清楚其现实意义。



参考文献

1. Bahri, Y., Dyer, E., Kaplan, J., et al. Explaining neural scaling laws. PNAS, 121(27), 2024. DOI: 10.1073/pnas.2311878121.
2. Hoffmann, J., Borgeaud, S., Mensch, A., et al. Training Compute-Optimal Large Language Models. NeurIPS 35, 2022.
3. Liu, Q., Zheng, X., Muennighoff, N., et al. RegMix: Data Mixture as Regression for Language Model Pre-training. ICLR 2025.
4. Biderman, S., Schoelkopf, H., Anthony, Q., et al. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. ICML 2023.
5. Asai, A., He, J., Shao, R., et al. Synthesizing scientific literature with retrieval-augmented language models. Nature 650, 857–863 (2026). DOI: 10.1038/s41586-025-10072-4.
6. Schaeffer, R., Miranda, B., Koyejo, S. Are Emergent Abilities of Large Language Models a Mirage? NeurIPS 36, 2023.
7. DeepSeek，元宝, WorrkBuddy. 数字整理，代码调试.

附录 A 附件清单

名称	修改日期	类型	大小
A1_quality_corrected.csv	2026/9/26 16:06	Microsoft Excel 逗号分隔值文件	3,029 KB
A1_quality_scores.csv	2026/9/26 20:42	Microsoft Excel 逗号分隔值文件	2,907 KB
A2_quality_corrected.csv	2026/9/26 16:06	Microsoft Excel 逗号分隔值文件	955 KB
A2_quality_scores.csv	2026/9/26 20:42	Microsoft Excel 逗号分隔值文件	942 KB
A3_quality_corrected.csv	2026/9/26 16:06	Microsoft Excel 逗号分隔值文件	11,402 KB
A3_quality_scores.csv	2026/9/26 20:43	Microsoft Excel 逗号分隔值文件	11,143 KB
conflict_summary.csv	2026/9/26 16:06	Microsoft Excel 逗号分隔值文件	5 KB
problem1_3_solution.csv	2026/9/27 8:54	Microsoft Excel 逗号分隔值文件	1 KB
problem2_solution.csv	2026/9/27 9:03	Microsoft Excel 逗号分隔值文件	1 KB
problem3_sensitivity_results.csv	2026/9/26 23:18	Microsoft Excel 逗号分隔值文件	57 KB
problem3_solution.csv	2026/9/27 7:39	Microsoft Excel 逗号分隔值文件	3 KB

附录 B 核心源码

Step1.py

```
from __future__ import annotations


import csv

import hashlib

import json

import lzma

import math

from collections import Counter, defaultdict

from pathlib import Path



ROOT = Path(__file__).resolve().parent

DATA = ROOT / "real_attachments" / "A_data_value"
```

```
TABLES = DATA / "regmix_tables"
OUT = ROOT / "step1_audit.json"


QUALITY_FIELDS = (
    "ad_en", "fluency_en", "modernbert_cleanliness",
    "modernbert_readability", "modernbert_reasoning",
    "modernbert_professionalism", "fineweb_edu", "qurater",
    "dsir_books", "dsir_wiki", "dsir_math",
    "rps_lines_ending_with_terminal_punctution_mark",
    "rps_doc_num_sentences", "rps_doc_word_count",
    "rps_doc_frac_no_alph_words", "rps_doc_frac_chars_top_2gram",
    "rps_lines_uppercase_letter_fraction", "rps_doc_frac_unique_words",
    "rps_lines_numerical_chars_fraction", "rps_doc_mean_word_length",
    "rps_doc_frac_chars_top_3gram", "rps_doc_unigram_entropy",
)
EXPECTED_LIST_LENGTHS = {
    "ad_en": 2, "fluency_en": 2,
    "modernbert_cleanliness": 6, "modernbert_readability": 6,
    "modernbert_reasoning": 6, "modernbert_professionalism": 6,
    "fineweb_edu": 1, "qurater": 4,
}


QUALITY_FILES = {
    "A1": DATA / "slimpajama_quality_signal_sample.jsonl.xz",
    "A2":                              next((DATA                              /
"slimpajama_quality_extended").glob("arxiv_*.jsonl.xz")),
    "A3":                              next((DATA                              /
"slimpajama_quality_extended").glob("github_*.jsonl.xz")),
}
```

```
CSV_FILES = {
    "A4": "train_mixture_1m.csv",
    "A5": "train_pile_loss_1m.csv",
    "A6": "test_mixture_1m.csv",
    "A7": "test_pile_loss_1m.csv",
    "A8": "test_mixture_60m.csv",
    "A9": "test_pile_loss_60m.csv",
    "A10": "test_mixture_1B.csv",
    "A11": "test_pile_loss_1B.csv",
    "A12": "est_mixture_10b.csv",
    "A13": "est_pile_loss_10b.csv",
    "A14": "est_mixture_70b.csv",
    "A15": "est_pile_loss_70b.csv",
}



def rel(path: Path) -> str:
    return path.relative_to(ROOT).as_posix()



def digest(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            h.update(block)
    return h.hexdigest()
```

```
def scan_quality(path: Path) -> tuple[dict, dict[str, str]]:
    result = {
        "path": rel(path), "compressed_bytes": path.stat().st_size,
        "records": 0, "parse_errors": 0, "non_object_records": 0,
        "duplicate_ids": 0, "empty_ids": 0,
        "field_presence": Counter(), "field_types": defaultdict(Counter),
        "null_or_empty": Counter(), "list_lengths": defaultdict(Counter),
        "nonfinite_values": Counter(), "bad_quality_shape": Counter(),
        "numeric_minmax": {}, "source_domains": Counter(),
        "nonempty_content": 0, "nonempty_sub_path": 0,
        "quality_field_count": len(QUALITY_FIELDS),
        "nonfinite_records": 0,
        "first_error_lines": [],
    }
    seen: dict[str, str] = {}
    with lzma.open(path, "rt", encoding="utf-8") as stream:
        for line_number, line in enumerate(stream, 1):
            try:
                row = json.loads(line)
            except (json.JSONDecodeError, UnicodeError) as exc:
                result["parse_errors"] += 1
                if len(result["first_error_lines"]) < 5:
                    result["first_error_lines"].append([line_number,
str(exc)[:100]])
                continue
            if not isinstance(row, dict):
                result["non_object_records"] += 1
                continue
```

```
            result["records"] += 1
            ident = row.get("id")
            if not isinstance(ident, str) or not ident:
                result["empty_ids"] += 1
            else:
                signal = {name: row.get(name) for name in QUALITY_FIELDS}
                signal_hash = hashlib.sha256(
                    json.dumps(signal,                          sort_keys=True,
allow_nan=True).encode("utf-8")
                ).hexdigest()
                if ident in seen:
                    result["duplicate_ids"] += 1
                else:
                    seen[ident] = signal_hash
            if row.get("content"):
                result["nonempty_content"] += 1
            if row.get("sub_path"):
                result["nonempty_sub_path"] += 1
            if row.get("_source_domain"):
                result["source_domains"][str(row["_source_domain"])] += 1
            for key, value in row.items():
                result["field_presence"][key] += 1
                result["field_types"][key][type(value).__name__] += 1
                if value is None or value == "" or value == []:
                    result["null_or_empty"][key] += 1
                if isinstance(value, list):
                    result["list_lengths"][key][str(len(value))] += 1
            record_has_nonfinite = False
            for key in QUALITY_FIELDS:
```

```
                value = row.get(key)
                if value is None:
                    result["bad_quality_shape"][key] += 1
                    continue
                if key in EXPECTED_LIST_LENGTHS:
                    if not isinstance(value, list) or len(value) != EXPECTED_LIST_LENGTHS[key]:
                        result["bad_quality_shape"][key] += 1
                elif isinstance(value, list):
                    result["bad_quality_shape"][key] += 1
                values = value if isinstance(value, list) else [value]
                if isinstance(value, list) and (not values or any(isinstance(x, (list, dict)) for x in values)):
                    result["bad_quality_shape"][key] += 1
                for x in values:
                    try:
                        number = float(x)
                    except (TypeError, ValueError, OverflowError):
                        result["bad_quality_shape"][key] += 1
                        continue
                    if not math.isfinite(number):
                        result["nonfinite_values"][key] += 1
                        record_has_nonfinite = True
                        continue
                    if key not in result["numeric_minmax"]:
                        result["numeric_minmax"][key] = [number, number]
                    else:
                        bounds = result["numeric_minmax"][key]
                        bounds[0] = min(bounds[0], number)
```

```
                        bounds[1] = max(bounds[1], number)
            result["nonfinite_records"] += record_has_nonfinite
    result["missing_quality_fields"] = {
        key: result["records"] - result["field_presence"].get(key, 0)
        for key in QUALITY_FIELDS
        if result["records"] != result["field_presence"].get(key, 0)
    }
    result["field_types"] = {k: dict(v) for k, v in result["field_types"].items()}
    result["list_lengths"] = {k: dict(v) for k, v in result["list_lengths"].items()}
    for key in ("field_presence", "null_or_empty", "nonfinite_values",
                "bad_quality_shape", "source_domains"):
        result[key] = dict(result[key])
    return result, seen



def scan_csv(path: Path, kind: str) -> tuple[dict, dict[str, tuple[str, ...]]]:
    result = {
        "path": rel(path), "bytes": path.stat().st_size,
        "sha256": digest(path), "kind": kind, "rows": 0,
        "columns": [], "value_columns": [], "duplicate_indices": 0,
        "missing_indices": 0, "empty_values": Counter(),
        "invalid_values": Counter(), "nonfinite_values": Counter(),
        "negative_values": Counter(), "value_minmax": {},
    }
    if kind == "mixture":
        result.update({"sum_minmax": None, "sum_outside_0_005": 0,
                       "active_domain_minmax":                    None,
```

```
"any_component_over_one": 0})
    rows_by_index: dict[str, tuple[str, ...]] = {}
    with path.open("r", encoding="utf-8-sig", newline="") as stream:
        reader = csv.DictReader(stream)
        result["columns"] = reader.fieldnames or []
        result["value_columns"] = [c for c in result["columns"] if c != "index"]
        for row in reader:
            result["rows"] += 1
            ident = row.get("index", "")
            if not ident:
                result["missing_indices"] += 1
            elif ident in rows_by_index:
                result["duplicate_indices"] += 1
            else:
                rows_by_index[ident] = tuple(row.get(c, "") for c in
result["value_columns"])
            numbers = []
            for key in result["value_columns"]:
                value = row.get(key)
                if value is None or value == "":
                    result["empty_values"][key] += 1
                    continue
                try:
                    number = float(value)
                except (TypeError, ValueError, OverflowError):
                    result["invalid_values"][key] += 1
                    continue
                if not math.isfinite(number):
                    result["nonfinite_values"][key] += 1
```

```
                    continue
                if number < 0:
                    result["negative_values"][key] += 1
                numbers.append(number)
                if key not in result["value_minmax"]:
                    result["value_minmax"][key] = [number, number]
                else:
                    bounds = result["value_minmax"][key]
                    bounds[0] = min(bounds[0], number)
                    bounds[1] = max(bounds[1], number)
            if kind == "mixture" and len(numbers) == len(result["value_columns"]):
                total = sum(numbers)
                active = sum(x > 0 for x in numbers)
                for key, x in (("sum_minmax", total), ("active_domain_minmax", active)):
                    bounds = result[key]
                    result[key] = [x, x] if bounds is None else [min(bounds[0], x), max(bounds[1], x)]
                result["sum_outside_0_005"] += abs(total - 1.0) > 0.005
                result["any_component_over_one"] += any(x > 1.0 for x in numbers)
    for key in ("empty_values", "invalid_values", "nonfinite_values", "negative_values"):
        result[key] = dict(result[key])
    return result, rows_by_index



def main() -> None:
    if len(QUALITY_FIELDS) != 22:
```

```
        raise AssertionError("Expected exactly 22 quality fields")
    report: dict = {"scope": "A1–A16 full read-only audit", "quality": {},
                    "tables": {}, "pairs": {}, "relationships": {}, "mapping":
{}}
    quality_ids = {}
    for name, path in QUALITY_FILES.items():
        print(f"Scanning {name}: {rel(path)}", flush=True)
        report["quality"][name], quality_ids[name] = scan_quality(path)
    for left, right in (("A1", "A2"), ("A1", "A3"), ("A2", "A3")):
        common = set(quality_ids[left]) & set(quality_ids[right])
        report["relationships"][f"{left}_{right}_id_overlap"] = len(common)
        report["relationships"][f"{left}_{right}_matching_quality_values"]  =
sum(
            quality_ids[left][i] == quality_ids[right][i] for i in common
        )
    report["relationships"]["total_records"]  =  sum(v["records"]  for  v  in
report["quality"].values())
    report["relationships"]["unique_ids"] = len(set().union(*(set(x) for x in
quality_ids.values())))


    table_rows = {}
    for name, filename in CSV_FILES.items():
        path = TABLES / filename
        kind = "mixture" if "mixture" in filename else "loss"
        report["tables"][name], table_rows[name] = scan_csv(path, kind)
    for mix, loss in (("A4", "A5"), ("A6", "A7"), ("A8", "A9"),
                      ("A10", "A11"), ("A12", "A13"), ("A14", "A15")):
        x, y = set(table_rows[mix]), set(table_rows[loss])
        report["pairs"][f"{mix}_{loss}"] = {
            "common_indices": len(x & y), "mixture_only": len(x - y),
```

```
            "loss_only": len(y - x),
        }
    for left, right in (("A6", "A8"), ("A12", "A14")):
        report["relationships"][f"{left}_{right}_same_file_bytes"] = (
            report["tables"][left]["sha256"]                                    ==
report["tables"][right]["sha256"]
        )
    mixture_tables = ("A4", "A6", "A8", "A10", "A12", "A14")
    for name in mixture_tables:
        report["tables"][name]["unique_recipe_vectors"]                         =
len(set(table_rows[name].values()))
    for left, right in (("A4", "A6"), ("A4", "A10"), ("A4", "A12"),
                        ("A6", "A10"), ("A6", "A12"), ("A10", "A12")):
        report["relationships"][f"{left}_{right}_recipe_overlap"] = len(
            set(table_rows[left].values()) & set(table_rows[right].values())
        )
    for left, right in (("A12", "A4"), ("A14", "A4")):
        shared = set(table_rows[left]) & set(table_rows[right])
        report["relationships"][f"{left}_{right}_shared_indices"]               =
len(shared)

report["relationships"][f"{left}_{right}_same_recipe_on_shared_index"] = sum(
            table_rows[left][i] == table_rows[right][i] for i in shared
        )
    mixture_cols = [set(report["tables"][x]["value_columns"]) for x in ("A4",
"A6", "A8", "A10", "A12", "A14")]
    loss_cols = [set(report["tables"][x]["value_columns"]) for x in ("A5", "A7",
"A9", "A11", "A13", "A15")]
    report["relationships"]["all_mixture_columns_agree"]    =    all(x    ==
mixture_cols[0] for x in mixture_cols)
    report["relationships"]["all_loss_columns_agree"] = all(x == loss_cols[0]
```

```
for x in loss_cols)
    train_domains = sorted(c.removeprefix("train_the_pile_") for c in mixture_cols[0])
    loss_domains = sorted(c.removeprefix("metric/the_pile_").removesuffix("_val_loss") for c in loss_cols[0])
    report["relationships"].update({"mixture_domains": train_domains,
                                    "loss_domains": loss_domains,
                                    "mixture_without_loss": sorted(set(train_domains) - set(loss_domains))})

    mapping_path = DATA / "domain_mapping_guide.csv"
    with mapping_path.open("r", encoding="utf-8-sig", newline="") as stream:
        mapping_rows = list(csv.DictReader(stream))
    report["mapping"] = {
        "path": rel(mapping_path), "rows": len(mapping_rows),
        "columns": list(mapping_rows[0]) if mapping_rows else [],
        "types": dict(Counter(x.get("mapping_type", "") for x in mapping_rows)),
        "missing_mixture_domains": sorted(set(train_domains) - {x["mixture_domain"] for x in mapping_rows}),
        "extra_mixture_domains": sorted({x["mixture_domain"] for x in mapping_rows} - set(train_domains)),
        "entries": [{k: row.get(k, "") for k in ("mixture_domain", "quality_domain", "mapping_type")}
                    for row in mapping_rows],
    }
    OUT.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(f"Wrote {OUT}", flush=True)
```

```
if __name__ == "__main__":
    main()
```

Step2.py

```
from __future__ import annotations


import csv
import json
import lzma
import math
from collections import Counter, defaultdict
from pathlib import Path


import numpy as np



ROOT = Path(__file__).resolve().parent
DATA = ROOT / "real_attachments" / "A_data_value"
OUTPUT = ROOT / "step2_outputs"
INPUTS = {
    "A1": DATA / "slimpajama_quality_signal_sample.jsonl.xz",
    "A2": next((DATA / "slimpajama_quality_extended").glob("arxiv_*.jsonl.xz")),
    "A3": next((DATA / "slimpajama_quality_extended").glob("github_*.jsonl.xz")),
}


FIELDS = (
```

```
    "ad_en", "fluency_en", "modernbert_cleanliness",
    "modernbert_readability", "modernbert_reasoning",
    "modernbert_professionalism", "fineweb_edu", "qurater",
    "dsir_books", "dsir_wiki", "dsir_math",
    "rps_lines_ending_with_terminal_punctution_mark",
    "rps_doc_num_sentences", "rps_doc_word_count",
    "rps_doc_frac_no_alph_words", "rps_doc_frac_chars_top_2gram",
    "rps_lines_uppercase_letter_fraction", "rps_doc_frac_unique_words",
    "rps_lines_numerical_chars_fraction", "rps_doc_mean_word_length",
    "rps_doc_frac_chars_top_3gram", "rps_doc_unigram_entropy",
)
RPS = tuple(x for x in FIELDS if x.startswith("rps_"))
DSIR = ("dsir_books", "dsir_wiki", "dsir_math")
MODERNBERT = tuple(x for x in FIELDS if x.startswith("modernbert_"))
QU_COMPONENTS = ("writing_style", "required_expertise", "facts_trivia",
"educational_value")
ROBUST_FIELDS = {"fineweb_edu", *DSIR,
                 "rps_doc_frac_chars_top_2gram",
"rps_doc_frac_chars_top_3gram"}



def finite_float(x: object) -> float: #把异常值变成缺失值
    try:
        value = float(x)
        return value if math.isfinite(value) else math.nan
    except (TypeError, ValueError, OverflowError):
        return math.nan
```

```
def binary_probability(logits: object, positive_index: int) -> float: #把两个 logit 转成正类概率
    """Numerically stable 2-class softmax probability."""
    if not isinstance(logits, list) or len(logits) != 2:
        return math.nan
    a, b = (finite_float(x) for x in logits)
    if not math.isfinite(a) or not math.isfinite(b):
        return math.nan
    delta = (b - a) if positive_index == 1 else (a - b)
    if delta >= 0:
        return 1.0 / (1.0 + math.exp(-delta))
    e = math.exp(delta)
    return e / (1.0 + e)



def ordinal_expectation(logits: object) -> float: #把六个 logit 合成一个分数
    if not isinstance(logits, list) or len(logits) != 6:
        return math.nan
    values = [finite_float(v) for v in logits]
    if not all(math.isfinite(v) for v in values):
        return math.nan
    maximum = max(values)
    exp_values = [math.exp(v - maximum) for v in values]
    total = sum(exp_values)
    return sum(k * p for k, p in enumerate(exp_values)) / (5 * total)



def clip01(x: float) -> float:
    return min(1.0, max(0.0, x))
```

```
def affine01(x: float, lower: float, upper: float) -> float: #在两个界限之间线性换算为 0～1，超出则截断
    if not math.isfinite(x):
        return math.nan
    if upper <= lower:
        return 0.5
    return clip01((x - lower) / (upper - lower))



def robust_sigmoid(x: float, p: dict) -> float: #以 A1 中位数为中心，平滑映射到 0～1
    if not math.isfinite(x):
        return math.nan
    iqr = p["q75"] - p["q25"]
    if iqr <= 0:
        return 0.5
    t = (2 * math.log(3) / iqr) * (x - p["q50"])
    if t >= 0:
        return 1 / (1 + math.exp(-t))
    e = math.exp(t)
    return e / (1 + e)



def trapezoid(x: float, a: float, b: float, c: float, d: float) -> float: #太低、太高都扣分，中间范围得满分
    if not math.isfinite(x):
        return math.nan
```

```
    if x <= a or x >= d:
        return 0.0
    if b <= x <= c:
        return 1.0
    if x < b:
        return affine01(x, a, b)
    return 1.0 - affine01(x, c, d)



def stream(path: Path):
    with lzma.open(path, "rt", encoding="utf-8") as source:
        for line in source:
            yield json.loads(line)



def percentiles(values: list[float], points: tuple[int, ...]) -> dict[str, float]:
    finite = np.asarray([x for x in values if math.isfinite(x)], dtype=float)
    if not len(finite):
        raise ValueError("Cannot fit a calibration to an empty numeric field")
    return {f"q{p:02d}": float(np.quantile(finite, p / 100)) for p in points}



def fit_raw_calibration(a1_rows: list[dict]) -> dict:
    columns = defaultdict(list)
    for row in a1_rows:
        for key in RPS:
            columns[key].append(finite_float(row[key]))
        for i, label in enumerate(QU_COMPONENTS):
```

```
            columns["qurater_"                                                    +
label].append(finite_float(row["qurater"][i]))

    return {key: percentiles(values, (1, 10, 25, 50, 75, 90, 99))

            for key, values in columns.items()}




def semantic_values(row: dict, raw_cal: dict) -> list[float]:

    q = row["qurater"]

    qu_scaled = [robust_sigmoid(finite_float(q[i]), raw_cal["qurater_" + name])

                 for i, name in enumerate(QU_COMPONENTS)]

    qu_value = sum(qu_scaled) / 4 if all(math.isfinite(x) for x in qu_scaled)
else math.nan

    words = finite_float(row["rps_doc_word_count"])

    dsir_per_word = {

        key: finite_float(row[key]) / words if words > 0 else math.nan

        for key in DSIR

    }

    r = {key: finite_float(row[key]) for key in RPS}

    c = raw_cal

    values = {

        "ad_en": binary_probability(row["ad_en"], 1),

        "fluency_en": binary_probability(row["fluency_en"], 1),

        "fineweb_edu": finite_float(row["fineweb_edu"][0]),

        "qurater": qu_value,

        **dsir_per_word,

    }

    for key in MODERNBERT:

        values[key] = ordinal_expectation(row[key])

    key = "rps_lines_ending_with_terminal_punctution_mark"
```

```
    values[key] = clip01(r[key] / 100) if math.isfinite(r[key]) else math.nan
    for key in ("rps_doc_num_sentences", "rps_doc_word_count"):
        # 过短受罚，到 A1 中位数后不再因长度增加而奖励。
        x = math.log1p(r[key]) if r[key] > -1 else math.nan
        values[key] = affine01(x, math.log1p(c[key]["q01"]),
                               math.log1p(c[key]["q50"]))
    for key in ("rps_doc_mean_word_length", "rps_doc_frac_unique_words",
                "rps_doc_unigram_entropy"):
        p = c[key]
        values[key] = trapezoid(r[key], p["q01"], p["q10"], p["q90"], p["q99"])
    for key in ("rps_doc_frac_no_alph_words",
                "rps_lines_uppercase_letter_fraction",
                "rps_lines_numerical_chars_fraction"):
        p = c[key]
        values[key] = 1 - affine01(r[key], p["q75"], p["q99"])
    for key in ("rps_doc_frac_chars_top_2gram",
"rps_doc_frac_chars_top_3gram"):
        # 高频片段覆盖越高，重复风险越高；超过 100 的原值只在最后被截到 0。
        values[key] = -r[key]
    return [values[key] for key in FIELDS]



def fit_final_calibration(semantic_rows: list[list[float]]) -> dict:
    columns = list(zip(*semantic_rows))
    fit = {}
    for key, col in zip(FIELDS, columns):
        fit[key] = percentiles(list(col), (1, 25, 50, 75, 99))
    return fit
```

```
def normalize(scores: list[float], fit: dict) -> tuple[list[float], list[str]]:
    output = []
    missing = []
    for key, value in zip(FIELDS, scores):
        if not math.isfinite(value):
            value = fit[key]["q50"]
            missing.append(key)
        if key in ROBUST_FIELDS:
            output.append(robust_sigmoid(value, fit[key]))
        else:
            output.append(value)
    return output, missing



def write_one(name: str, path: Path, raw_cal: dict, final_cal: dict,
              cached_rows: list[dict] | None = None,
              cached_semantic: list[list[float]] | None = None) -> dict:
    output_path = OUTPUT / f"{name}_22_scores.csv"
    summary = {"records": 0, "missing_semantic": Counter(),
               "at_zero": Counter(), "at_one": Counter(),
               "domain_counts": Counter(), "out_of_range": Counter()}
    if cached_rows is None:
        iterator = ((row, semantic_values(row, raw_cal)) for row in
stream(path))
    else:
        assert cached_semantic is not None
        iterator = zip(cached_rows, cached_semantic)
    with output_path.open("w", encoding="utf-8", newline="") as destination:
```

```
        writer = csv.writer(destination)
        writer.writerow(["id", "quality_domain", *FIELDS, "imputed_fields"])
        for row, scores in iterator:
            normalized, missing = normalize(scores, final_cal)
            domain = row.get("_source_domain") or {"A2": "arxiv", "A3": "github"}[name]
            writer.writerow([row["id"], domain,
                             *(f"{v:.12f}" for v in normalized), ";".join(missing)])
            summary["records"] += 1
            summary["domain_counts"][domain] += 1
            for key in missing:
                summary["missing_semantic"][key] += 1
            for key, raw_value, value in zip(FIELDS, scores, normalized):
                summary["at_zero"][key] += value == 0
                summary["at_one"][key] += value == 1
                if math.isfinite(raw_value):
                    summary["out_of_range"][key] += (
                        raw_value < final_cal[key]["q01"] or
                        raw_value > final_cal[key]["q99"]
                    )
    summary["path"] = output_path.relative_to(ROOT).as_posix()
    for key in ("missing_semantic", "at_zero", "at_one", "domain_counts", "out_of_range"):
        summary[key] = dict(summary[key])
    return summary



def main() -> None:
```

```
    if len(FIELDS) != 22:
        raise AssertionError("Expected 22 fields")
    OUTPUT.mkdir(exist_ok=True)
    print("Fitting A1 raw calibration", flush=True)
    # 原文仅用于后续人工验证；拟合时不把大段 content 留在内存。
    keep = set(FIELDS) | {"id", "_source_domain"}
    a1_rows = [{key: value for key, value in row.items() if key in keep}
               for row in stream(INPUTS["A1"])]
    raw_cal = fit_raw_calibration(a1_rows)
    print("Fitting A1 final calibration", flush=True)
    a1_semantic = [semantic_values(row, raw_cal) for row in a1_rows]
    final_cal = fit_final_calibration(a1_semantic)
    calibration = {
        "fit_source": "A1 only", "fit_records": len(a1_rows),
        "fields": list(FIELDS), "raw_quantiles": raw_cal,
        "final_quantiles": final_cal,
        "method": "logit probabilities/ordinal expectations kept in [0,1];
bounded heuristic utilities kept in [0,1]; unbounded fields use A1-median/IQR
robust sigmoid without clipping",
    }
    (OUTPUT / "calibration.json").write_text(
        json.dumps(calibration, ensure_ascii=False, indent=2, allow_nan=False)
+ "\n",
        encoding="utf-8",
    )
    summary = {}
    for name, path in INPUTS.items():
        print(f"Transforming {name}", flush=True)
        summary[name] = write_one(
```

```
            name, path, raw_cal, final_cal,
            a1_rows if name == "A1" else None,
            a1_semantic if name == "A1" else None,
        )
    (OUTPUT / "summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False) +
"\n",
        encoding="utf-8",
    )
    print(f"Wrote {OUTPUT}", flush=True)



if __name__ == "__main__":
    main()
```

Step3-1.py

```
from __future__ import annotations


import csv
import json
import math
from collections import defaultdict
from pathlib import Path


import numpy as np
from scipy.stats import spearmanr



ROOT = Path(__file__).resolve().parent
```

```
INPUT = ROOT / "step2_outputs"

OUTPUT = ROOT / "step3_outputs"

SOURCES = ("A1", "A2", "A3")



def read_scores(name: str, fields: list[str], expected_n: int):

    x = np.empty((expected_n, len(fields)), dtype=np.float64)

    ids: list[str] = []

    domains: list[str] = []

    imputed: list[str] = []

    path = INPUT / f"{name}_22_scores.csv"

    with path.open("r", encoding="utf-8", newline="") as stream:

        reader = csv.DictReader(stream)

        if reader.fieldnames != ["id", "quality_domain", *fields,
"imputed_fields"]:

            raise ValueError(f"Unexpected columns in {path}")

        for i, row in enumerate(reader):

            if i >= expected_n:

                raise ValueError(f"Too many rows in {path}")

            ids.append(row["id"])

            domains.append(row["quality_domain"])

            imputed.append(row["imputed_fields"])

            x[i] = [float(row[key]) for key in fields]

    if len(ids) != expected_n:

        raise ValueError(f"Expected {expected_n} rows, got {len(ids)} in
{path}")

    if not np.all(np.isfinite(x)) or np.any((x < 0) | (x > 1)):

        raise ValueError(f"Nonfinite or out-of-range scores in {path}")

    return ids, domains, imputed, x
```

```
def entropy_diversity(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    n = len(x)
    sums = x.sum(axis=0)
    if np.any(sums <= 0):
        raise ValueError("At least one score column sums to zero")
    p = x / sums
    log_p = np.zeros_like(p)
    np.log(p, out=log_p, where=p > 0)
    entropy = -np.sum(p * log_p, axis=0) / math.log(n)
    diversity = np.maximum(0, 1 - entropy)
    return entropy, diversity



def score_matrix(x: np.ndarray, weights: np.ndarray) -> dict[str, np.ndarray]:
    main = x @ weights
    weighted = x * weights
    d_good = np.linalg.norm(weights - weighted, axis=1)
    d_bad = np.linalg.norm(weighted, axis=1)
    topsis = d_bad / (d_good + d_bad)
    return {"Q_main": main, "Q_topsis": topsis}



def add_summary(buckets: dict, source: str, domain: str,
                values: tuple[float, ...]) -> None:
    buckets[(source, domain)].append(values)
```

```
def aggregate(buckets: dict, measure_names: tuple[str, ...]) -> list[dict]:
    rows = []
    for (source, domain), values in sorted(buckets.items()):
        a = np.asarray(values, dtype=float)
        row: dict = {"source": source, "quality_domain": domain, "n": len(a)}
        for j, name in enumerate(measure_names):
            row[f"{name}_mean"] = float(np.mean(a[:, j]))
            row[f"{name}_median"] = float(np.median(a[:, j]))
            row[f"{name}_q10"] = float(np.quantile(a[:, j], .1))
            row[f"{name}_q90"] = float(np.quantile(a[:, j], .9))
        rows.append(row)
    return rows



def main() -> None:
    OUTPUT.mkdir(exist_ok=True)
    calibration = json.loads((INPUT / "calibration.json").read_text(encoding="utf-8"))
    validation = json.loads((INPUT / "validation.json").read_text(encoding="utf-8"))
    fields = calibration["fields"]
    if len(fields) != 22:
        raise ValueError("Expected 22 fields")
    print("Fitting entropy diversity on A1", flush=True)
    _, _, _, a1_x = read_scores("A1", fields, validation["A1"]["rows"])
    entropy, diversity = entropy_diversity(a1_x)
    if diversity.sum() <= 0:
        raise ValueError("All indicators are constant; entropy weights are
```

```
undefined")
    weights = diversity / diversity.sum()


    with (OUTPUT / "weights.csv").open("w", encoding="utf-8", newline="") as
stream:
        writer = csv.writer(stream)
        writer.writerow(["field",        "A1_entropy",        "A1_diversity",
"final_weight"])
        for j, field in enumerate(fields):
            writer.writerow([field, f"{entropy[j]:.12f}",
                             f"{diversity[j]:.12f}", f"{weights[j]:.12f}"])
    model_spec = {
        "fit_source": "A1 only", "A1_n": len(a1_x),
        "weight_method": "direct entropy weights across all 22 indicators",
        "formula": "Q_main=sum_j(weights[j]*z[i,j])",
        "conflict_adjustment": "none; pending a new definition and rule",
        "weights": {field: float(weights[j]) for j, field in enumerate(fields)},
        "topsis_ideal": {"positive": 1, "negative": 0},
    }
    (OUTPUT / "model_spec.json").write_text(
        json.dumps(model_spec,    ensure_ascii=False,    indent=2)    +    "\n",
encoding="utf-8"
    )


    measure_names = ("Q_main", "Q_topsis")
    buckets: dict[tuple[str, str], list[tuple[float, ...]]] = defaultdict(list)
    seen_ids: set[str] = set()
    sensitivity = {}
    for name in SOURCES:
```

```
        print(f"Scoring {name}", flush=True)
        ids, domains, imputed, x = read_scores(name, fields, validation[name]["rows"])
        result = score_matrix(x, weights)
        variants = {"topsis": result["Q_topsis"]}
        sensitivity[name] = {
            key: float(spearmanr(result["Q_main"], value).statistic)
            for key, value in variants.items()
        }
        path = OUTPUT / f"{name}_quality_scores.csv"
        with path.open("w", encoding="utf-8", newline="") as stream:
            writer = csv.writer(stream)
            writer.writerow(["id", "quality_domain", *result.keys(), "imputed_fields"])
            for i, ident in enumerate(ids):
                writer.writerow([ident, domains[i],
                                 *(f"{result[key][i]:.10f}" for key in result),
                                 imputed[i]])
                values = tuple(float(result[key][i]) for key in measure_names)
                add_summary(buckets, name, domains[i], values)
                if ident not in seen_ids:
                    add_summary(buckets, "unique_union", domains[i], values)
                    seen_ids.add(ident)
    summary_rows = aggregate(buckets, measure_names)
    summary_path = OUTPUT / "domain_summary.csv"
    with summary_path.open("w", encoding="utf-8", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=list(summary_rows[0]))
        writer.writeheader()
```

```
        writer.writerows(summary_rows)
    report = {
        "unique_union_ids": len(seen_ids),
        "ranking_spearman_vs_main": sensitivity,
        "A1_A2_arxiv_mean_difference": next(row["Q_main_mean"] for row in
summary_rows
                                            if row["source"] == "A2" and
row["quality_domain"] == "arxiv")
                                    - next(row["Q_main_mean"] for row in
summary_rows
                                           if row["source"] == "A1" and
row["quality_domain"] == "arxiv"),
        "A1_A3_github_mean_difference": next(row["Q_main_mean"] for row in
summary_rows
                                             if row["source"] == "A3" and
row["quality_domain"] == "github")
                                     - next(row["Q_main_mean"] for row in
summary_rows
                                            if row["source"] == "A1" and
row["quality_domain"] == "github"),
    }
    (OUTPUT / "comparison.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2) + "\n",
encoding="utf-8"
    )
    print(f"Wrote {OUTPUT}", flush=True)



if __name__ == "__main__":
    main()
```

Step3-2.py

```
from __future__ import annotations


import csv

import json

import lzma

from collections import defaultdict

from pathlib import Path




ROOT = Path(__file__).resolve().parent

DATA = ROOT / "real_attachments" / "A_data_value"

OUTPUT = ROOT / "step3_outputs"

INPUTS = {

    "A1": DATA / "slimpajama_quality_signal_sample.jsonl.xz",

    "A2":                                next((DATA                                /
"slimpajama_quality_extended").glob("arxiv_*.jsonl.xz")),

    "A3":                                next((DATA                                /
"slimpajama_quality_extended").glob("github_*.jsonl.xz")),

}

MEASURES = ("Q_main",)




def add(acc: dict, source: str, domain: str, words: float, values: dict) -> None:

    item = acc[(source, domain)]

    item["n"] += 1

    item["sum_words"] += words

    item["sum_words_squared"] += words * words

    item["max_words"] = max(item["max_words"], words)

    for key in MEASURES:
```

```
        item[key + "_sum"] += values[key]
        item[key + "_word_sum"] += words * values[key]



def main() -> None:
    acc = defaultdict(lambda: defaultdict(float))
    seen_ids = set()
    for source, path in INPUTS.items():
        print(f"Aggregating {source}", flush=True)
        scores_path = OUTPUT / f"{source}_quality_scores.csv"
        with lzma.open(path, "rt", encoding="utf-8") as raw_stream, \
                scores_path.open("r",    encoding="utf-8",    newline="")    as
scores_stream:
            scores = csv.DictReader(scores_stream)
            count = 0
            for raw_line, score in zip(raw_stream, scores, strict=True):
                raw = json.loads(raw_line)
                if raw["id"] != score["id"]:
                    raise ValueError(f"Record order mismatch in {source} at
{count}")
                words = float(raw["rps_doc_word_count"])
                if words <= 0:
                    raise ValueError("Expected positive word count")
                domain = score["quality_domain"]
                values = {key: float(score[key]) for key in MEASURES}
                add(acc, source, domain, words, values)
                if raw["id"] not in seen_ids:
                    add(acc, "unique_union", domain, words, values)
                    seen_ids.add(raw["id"])
```

```
                count += 1
        print(f"  {count} rows", flush=True)
    rows = []
    for (source, domain), item in sorted(acc.items()):
        row = {"source": source, "quality_domain": domain, "n": int(item["n"]),
               "total_words_proxy": item["sum_words"],
               "mean_words": item["sum_words"] / item["n"],
               "max_word_share": item["max_words"] / item["sum_words"],
               "word_weight_effective_n":    item["sum_words"]    **    2    /
item["sum_words_squared"]}
        for key in MEASURES:
            row[key + "_document_mean"] = item[key + "_sum"] / item["n"]
            row[key  +  "_word_weighted_mean"]  =  item[key  +  "_word_sum"]  /
item["sum_words"]
            row[key + "_difference"] = (row[key + "_word_weighted_mean"]
                                        - row[key + "_document_mean"])
        rows.append(row)
    path = OUTPUT / "length_weighted_summary.csv"
    with path.open("w", encoding="utf-8", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    assert sum(row["n"] for row in rows if row["source"] == "unique_union") ==
261086
    print(f"Wrote {path}")



if __name__ == "__main__":
    main()
```